

Homework 4

ME 326: Collaborative Robotics

Stanford University

NOTE - Run this script with CUDA (GPU)

You are encouraged to run this script with GPU enabled. If you are using Google Colab, you can select a GPU runtime. By selecting `Runtime` then `Change session runtime` you can ensure GPU is the hardware accelerator [tutorial](#).

```
In [3]: # make sure to install the packages below!

from __future__ import print_function
import argparse
import torch
from torch import nn, optim
from torch.nn import functional as F
from torchvision import datasets, transforms
from torchvision.utils import save_image
import torch.utils.data

import numpy as np
import matplotlib
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import os #for making a results directory in problem 2
import math
import requests
from PIL import Image
from io import BytesIO
import torchvision.transforms as T
```

Readme:

This homework leverages an accompanying MNIST dataset. The following lines pull down this well-known dataset:

```
In [4]: import torchvision.datasets as datasets
mnist_trainset = datasets.MNIST(root='./data', train=True, download=True, tr
```

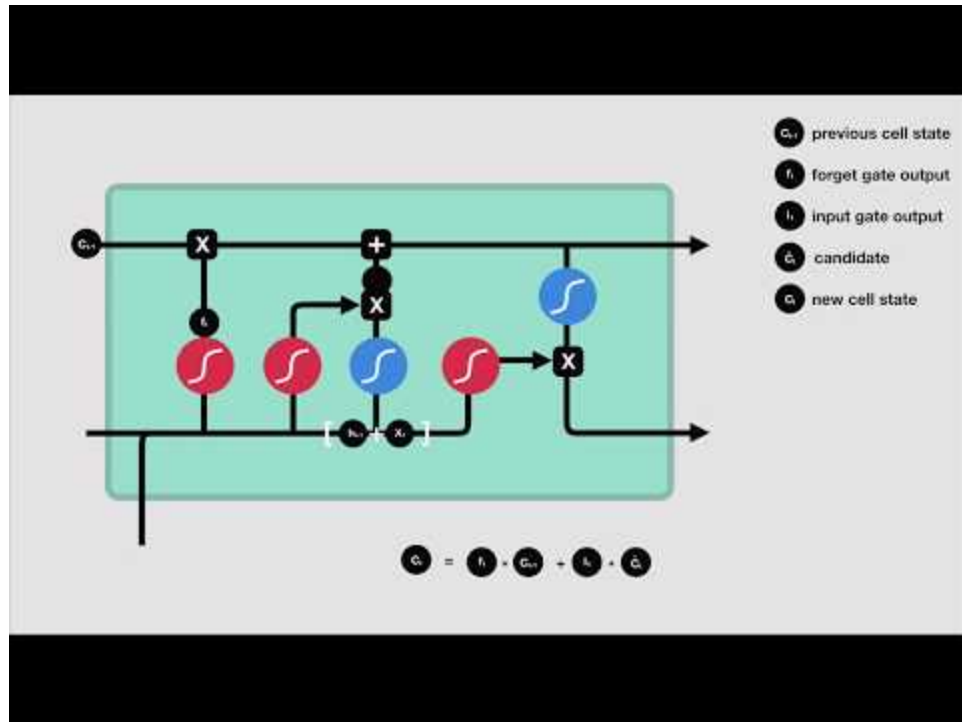
```
100%|██████████| 9.91M/9.91M [00:01<00:00, 5.01MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 135kB/s]
100%|██████████| 1.65M/1.65M [00:01<00:00, 1.26MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 12.3MB/s]
```

```
In [5]: data_path = 'data' #run this if you are running this locally and the data fo
```

Problem 1: LSTM Sequence Prediction

Figures in this problem are referenced/imported from this toward data science article: [Illustrated Guide to LSTM's and GRU's: A step by step explanation](#) (and is encouraged for further reading)

Check out this [video](#) for a video summary of how LSTM's work:



Sequential Data

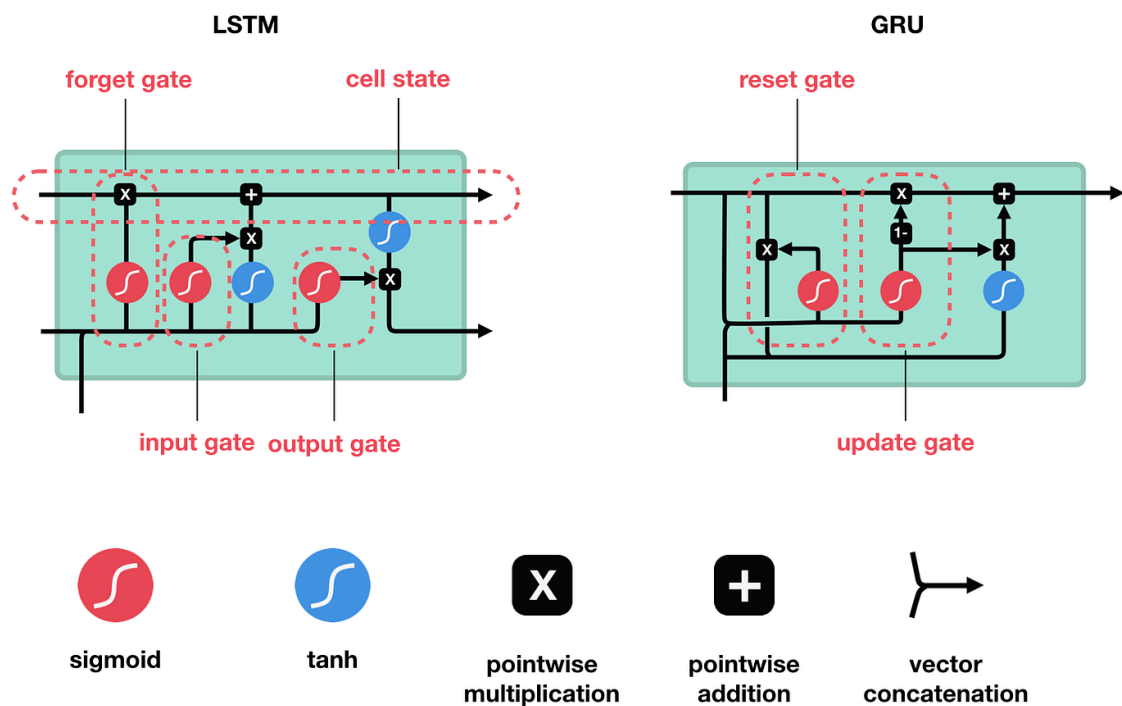
In some problems, it is beneficial to recognize a pattern that emerges from a sequence of data. If there are underlying principles driving behavior, they may be best interpreted by observing the system over time then using those observations to extrapolate to expected future sequence states.

This leads to the study of **Recurrent Neural Network**(RNN) which are designed specifically to work with sequential data (could be time dependent or time independent). The key components of the general architecture of a RNN is the input vector x (which was observed or known) being fed into the RNN, and the RNN maintains *hidden* states h which serve as "memory" for the network as it processes new input and/or predicts future outputs. A key challenge in such a network is the question: "what information is most important to remember?"

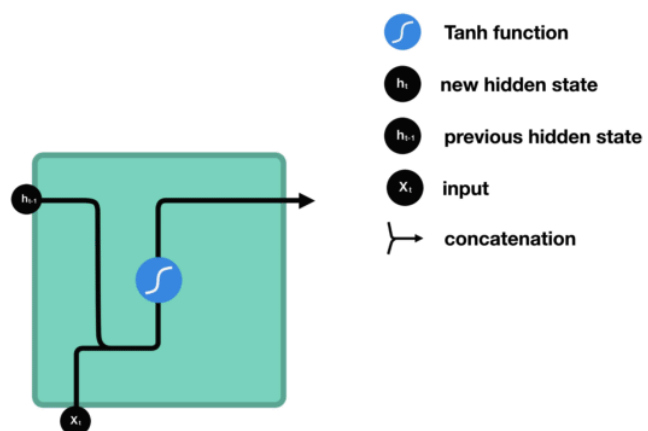
Long-Short Term Memory (LSTM) networks (a type of RNN) are one answer to this question, with the ability to *forget*, *input new information into memory*, provide both a *new output (hidden state)* available to external query as well as *maintain an updated internal state (cell state)* for future memory iterations.

While it will not be discussed further in this homework, the student should be aware of another option - [Attention Networks](#) which observe a sequence and determine what is the most important information to *pay attention to* in order to make some prediction or estimation.

For the LSTM, the structure is shown here and consists of a forget gate, cell state, input gate and output gate. For completeness, a Gated Recurrent Unit (GRU) is shown (very similar to LSTM with a few less 'control' handles).

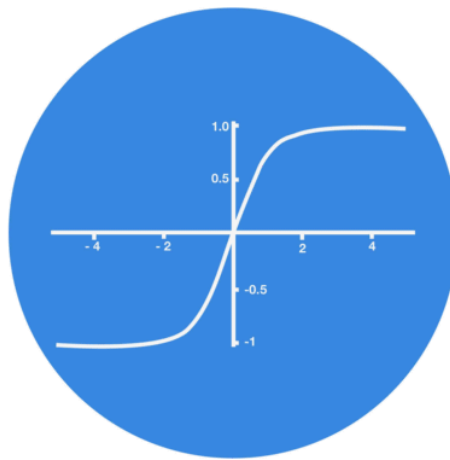


A basic RNN cell takes in a hidden state and input state, passes it through a Tanh function and outputs a new hidden state

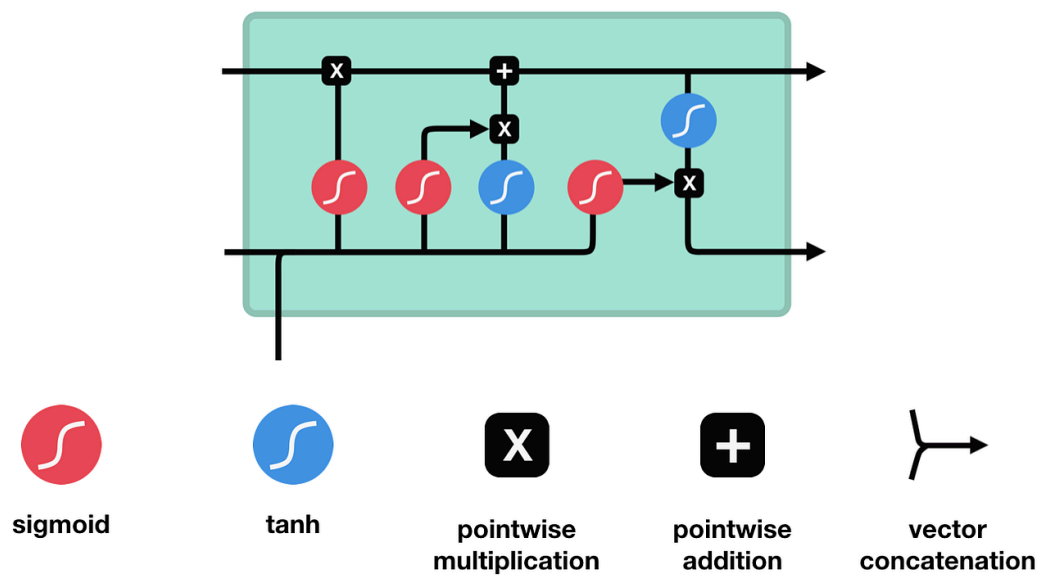


where the Tanh activation serves to map the values between -1 and 1

5
0.1
-0.5

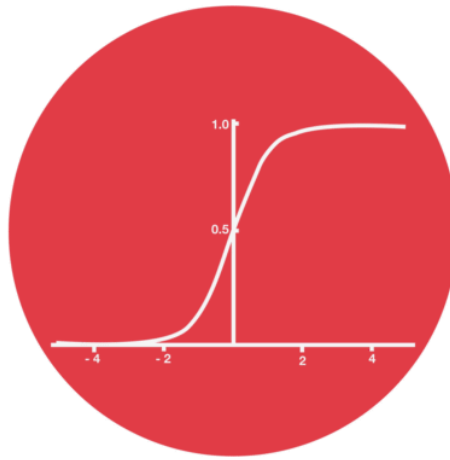


The architecture of an LSTM cell is shown below:



with the key attribute being its various gates. The *cell state* is designed to carry relevant information throughout iterations of the sequence serving as a 'transport highway' of information. As the cell state progresses, it must be informed by new information and focus on what is important through the use of *gates*. The *gates* leverage sigmoid activation functions which compress continuous input information between 0 and 1:

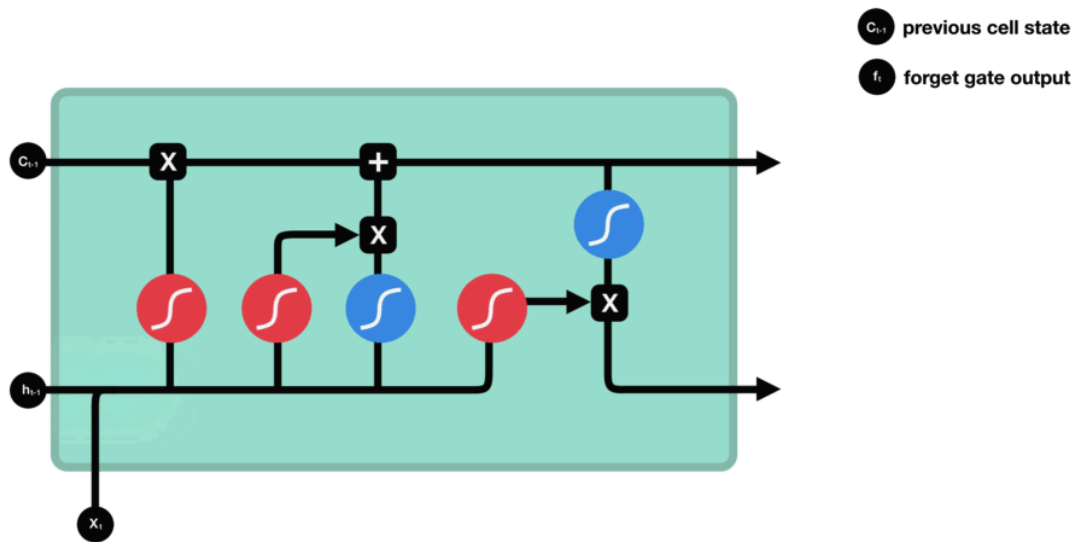
5
0.1
-0.5



The different gates are as follows:

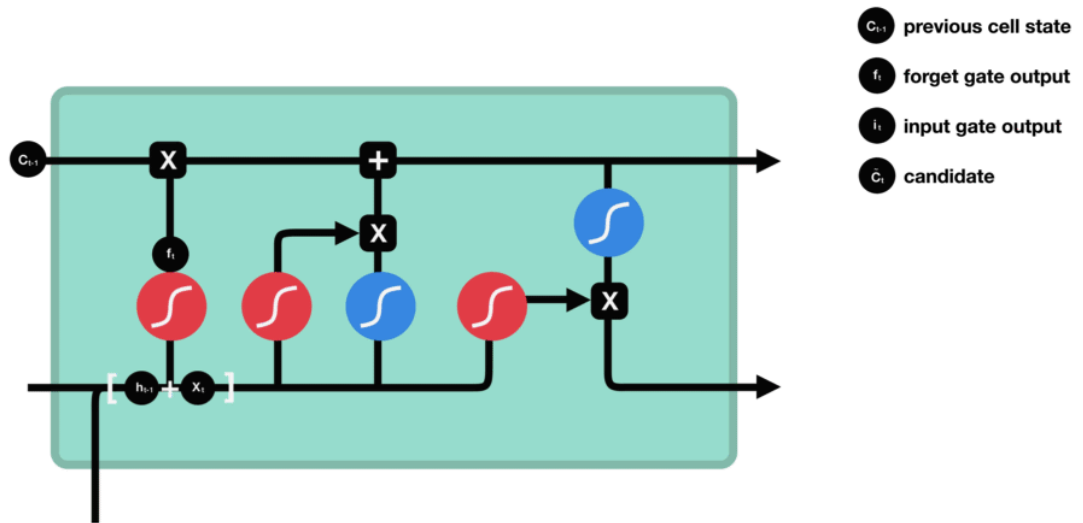
Forget Gate

The *forget gate* decides which information should be 'thrown away' or kept and this is done by passing added previous h and current x information through a sigmoid function where values closer to 0 implies to *forget* and values closer to 1 indicate to *keep*



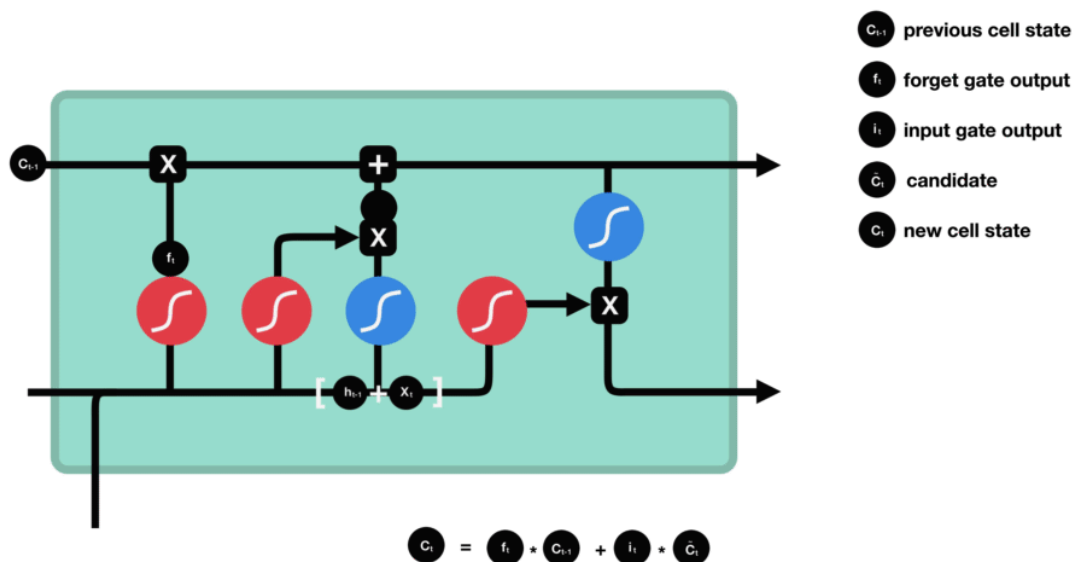
Input Gate

The *input gate* updates the cell state with new information. This is achieved by passing added hidden and current state through a sigmoid and tanh functions respectively then multiplying the outputs. The sigmoid function effectively determines how important each value is (0 non-important; 1 important). The tanh function serves to regulate the network by mapping the values between -1 and 1. Because it is a pointwise multiplication, the sigmoid directly determines which values to keep from the tanh output.



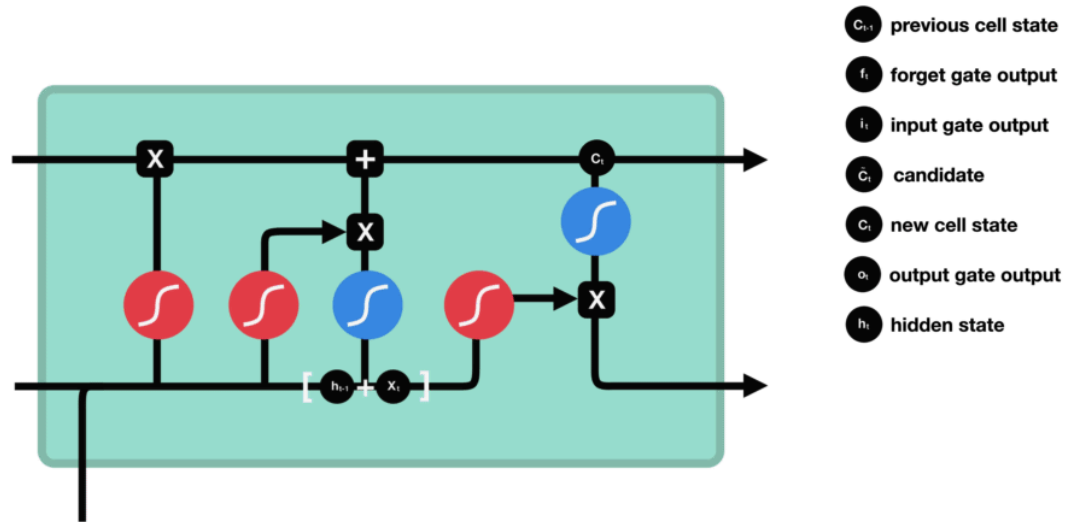
Cell State

Given outputs from the *forget* and *input gates* we can now update the internal *cell state* by first multiplying by the forget vector (which could possibly cause dropping of values in the cell state if forget vector has values close to zero). Then the output from the cell state and forget vector is then point-wise added to the outcome of the input gate which updates the cell state with values that the network deems relevant.



Output Gate

Finally, for the *output gate* which determines what the next hidden state will be, we must first take the previous hidden state and input and pass it through a sigmoid function (for importance) then multiply it by the newest cell state passed through a tanh function (regularized), and this determines what information the hidden state should carry. **The final output (to the world outside the LSTM cell) is the hidden state.**



In summary, the forget gate decides what's relevant from prior steps; the input gate decides what new information is relevant to add to the cell state and the output gate determines what the next hidden state should become.

Python Psuedo Code:

```
def LSTMCELL(prev_ct, prev_ht, input):
    combine = prev_ht + input
    ft = forget_layer(combine)
    candidate = candidate_layer(combine)
    it = input_layer(combine)
    Ct = prev_ct * ft + candidate * it
    ot = output_layer(combine)
    ht = ot * tanh(Ct)
    return ht, Ct
```

```
ct = [0, 0, 0]
ht = [0, 0, 0]
for input in inputs:
    ct, ht = LSTMCELL(ct, ht, input)
```

Pytorch [documentation on LSTM](#).

Your Problem: predicting a sequence

In this problem, you will predict the next values in a sequence function given history of values by training an LSTM.

Unlike previous scripts, your code will be to fill in components of a class that will be used by support code to track and predict a series of sine curves.

You will generate two LSTM cells whose hidden state dimension is 51 h_{51} , c_{51} , and your inputs will flow as follows:

$$x \rightarrow \text{LSTM}_1(x, (h_{1,t}, c_{1,t})) \rightarrow (h_{1,t}, c_{1,t}) \rightarrow \text{LSTM}_2(h_{1,t}, (h_{2,t}, c_{2,t})) \rightarrow (h_{2,t}, c_{2,t}) \rightarrow y$$

Your function for forward pass will take the training data, learning rate, training steps, and number of "future" steps to extrapolate into the future.

Your Code:

```
In [3]: def student_function_1(data,lr=0.8,training_steps = 5,future = 1000):
        """
        Description: This function implements a 2 cell LSTM model
        Input to whole model x-input -> Model -> y-output (where y = sin(x))
        Model architecture: x -> LSTM1(x,h1,c1)-> h1 -> LSTM2(h1,h2,c2) -> h2 ->

        Parameters
        -----
        data : torch Tensor
                input data x points
        training_steps : int
                number of steps to train the model -- you can reduce this number!
        future : int
                number of sequences to predict in future

        Returns
        -----
        output : np.matrix
                output
        """
        # raise NotImplementedError #comment this out once implemented

        class Sequence(nn.Module):
            def __init__(self,hidden_state_size=51):
                super(Sequence, self).__init__()
                self.hidden_state_size = hidden_state_size
                # Define the shape of the two-cell network
                #First LSTM Cell (input dim 1; output dim: self.hidden_state_size)
                self.lstm1 = nn.LSTMCell(1, self.hidden_state_size) #Fill this in
                #Second LSTM cell (input dim - size of lstm1 output (self.hidden_state_size))
                self.lstm2 = nn.LSTMCell(self.hidden_state_size, self.hidden_state_size)
                #output of the second LSTM needs to be passed through a linear layer
                self.linear = nn.Linear(self.hidden_state_size, 1) #Fill this in

            def forward(self, input, future = 0):
                #make a running list of 'outputs':
                outputs = []
                #For forward pass of the network, instantiate the hidden and cell states
                h_t = torch.zeros(input.size(0), self.hidden_state_size, dtype=torch.float)
                c_t = torch.zeros(input.size(0), self.hidden_state_size, dtype=torch.float)
                h_t2 = torch.zeros(input.size(0), self.hidden_state_size, dtype=torch.float)
                c_t2 = torch.zeros(input.size(0), self.hidden_state_size, dtype=torch.float)
                #Now define the forward pass of the network
                for input_t in input.split(1, dim=1):
                    #Perform a forward pass through the first LSTM cell, input is input_t
                    #Fill this in:
```



```

        h_t, c_t = self.lstm1(input_t, (h_t, c_t))

        #Perform a forward pass through the second LSTM cell, input
        #Fill this in:
        h_t2, c_t2 = self.lstm2(h_t, (h_t2, c_t2))
        #Perform linear forward pass, input is h_t2; Output is 'output'
        #Fill this in:
        output = self.linear(h_t2)
        #add output to the list of 'outputs'
        outputs += [output]
    for i in range(future):# if we should predict the future
        '''
        Perform forward pass, starting with LSTM 1 cell, its input is
        output from above 'output' and (h_t,c_t);
        '''
        #Fill this in:
        h_t, c_t = self.lstm1(output, (h_t, c_t))
        #Perform forward pass through the second LSTM cell, inputs h
        #Fill this in:
        h_t2, c_t2 = self.lstm2(h_t, (h_t2, c_t2))
        #Now pass the hidden layer from the second LSTM cell through
        #Fill this in:
        output = self.linear(h_t2)
        outputs.append(output)
    # Now that all passes through the LSTM cells are complete we have
    outputs = torch.cat(outputs, dim=1)
    return outputs

```

```

# START HERE: here is where we actually build the model; we will referen
np.random.seed(0) #this is important to randomly initate weights for tr
torch.manual_seed(0) #this specifically sets seed for generating random
input = torch.from_numpy(data[3:, :-1]) #Inputs: convert the data passec
target = torch.from_numpy(data[3:, 1:]) #Supervised target (labels for h
'''

```

```

Test input/output will NOT be exposed to our model, this is used for us
'''

```

```

test_input = torch.from_numpy(data[:3, :-1])
test_target = torch.from_numpy(data[:3, 1:])
# build the model
seq = Sequence()
seq.double()
criterion = nn.MSELoss()
# use LBFGS as optimizer since we can load the whole data to train
optimizer = optim.LBFGS(seq.parameters(), lr=lr)
#begin to train
for i in range(training_steps):
    print('STEP: ', i)
    def closure():
        optimizer.zero_grad()
        out = seq(input)
        loss = criterion(out, target)
        print('loss:', loss.item())
        loss.backward()
    return loss

```

```

optimizer.step(closure)
# begin to predict, no need to track gradient here
with torch.no_grad():
    future = 1000
    pred = seq(test_input, future=future)
    loss = criterion(pred[:, :-future], test_target)
    print('test loss:', loss.item())
    y = pred.detach().numpy()
# draw the result
plt.figure(figsize=(30,10))
plt.title('Predict future values for time sequences\n(Dashlines are
plt.xlabel('x', fontsize=20)
plt.ylabel('y', fontsize=20)
plt.xticks(fontsize=20)
plt.yticks(fontsize=20)
def draw(yi, color):
    plt.plot(np.arange(input.size(1)), yi[:input.size(1)], color, li
    plt.plot(np.arange(input.size(1), input.size(1) + future), yi[ir
draw(y[0], 'r')
draw(y[1], 'g')
draw(y[2], 'b')
plt.savefig('predict%d.pdf'%i)
plt.close()

#return the final prediction
return y

```

Evaluation

```

In [4]: #generate data:
np.random.seed(2)
T = 20
L = 1000
N = 100
future = 1000
'''
We will generate 100 sequences that each have 1000 sequential elements in th
We will use the np.random.randint(...) to have random starts for the sequenc
but only 1 randomly generated integer is subtracted from an entire list - wh
the only variable is where you start counting (anywhere from -4 to 4)
'''
x = np.empty((N, L), 'int64')
x[:, :] = np.array(range(L)) + np.random.randint(-4, 4, N).reshape(N, 1)
'''
Now that we have 100 sequences of x-values (that count monotonically from th
lets now pass all of these x-values through a sine function to get the 'data
'''
data = np.sin(x / 1.0 / T).astype('float64')
#YOUR CODE IS CALLED HERE AND PASSED THE DATA and number of future steps to
y = student_function_1(data, future = 1000)

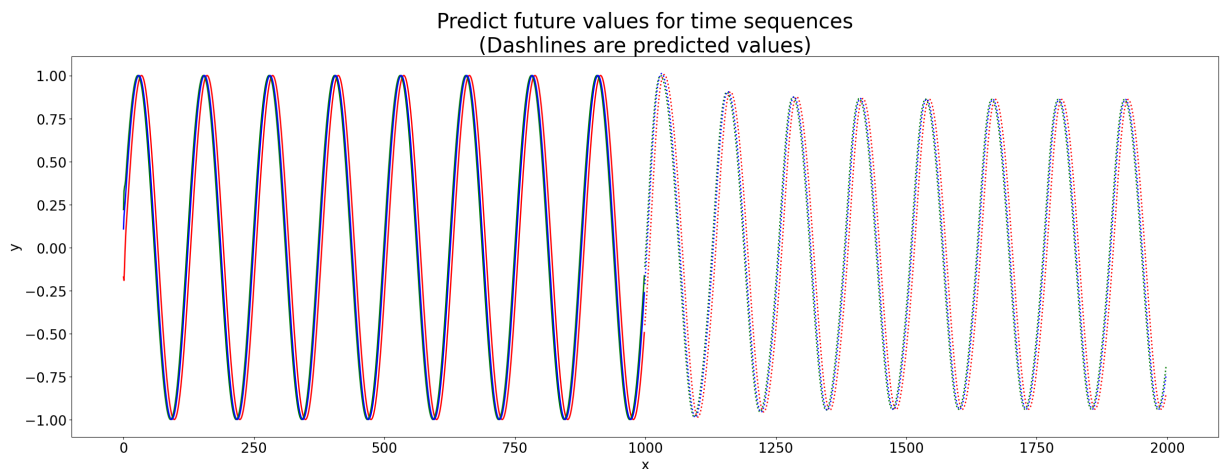
```

STEP: 0
loss: 0.505131989258659
loss: 0.5013177301966176
loss: 0.4813109290806786
loss: 0.44775423343714305
loss: 0.36262684284005153
loss: 0.20862141819820776
loss: 0.6098807484581915
loss: 0.029624599963305936
loss: 0.029072787135558296
loss: 0.025639129561867615
loss: 0.02220569401936699
loss: 0.016729062070172114
loss: 0.011932162762423317
loss: 0.011018334434944823
loss: 0.008367864882735351
loss: 0.005329313794595773
loss: 0.003074497739644866
loss: 0.0013660558970532877
loss: 0.0009542847616437017
loss: 0.0008982725874896632
test loss: 0.000815691119197325
STEP: 1
loss: 0.0008249418204703195
loss: 0.0007877275248704695
loss: 0.0007277226112016323
loss: 0.0006560909283159157
loss: 0.0006046536920005349
loss: 0.0005815339953593595
loss: 0.0005737982348862304
loss: 0.0005708666283802858
loss: 0.0005658835990122502
loss: 0.0005530472369349588
loss: 0.0005258930090212729
loss: 0.0004702872089384662
loss: 0.0003770649944575564
loss: 0.00025926690608402754
loss: 0.00013679517668454847
loss: 0.00013722133678943025
loss: 0.00011263367171022061
loss: 0.00010757242905182339
loss: 0.00010332356414529842
loss: 9.89670289579655e-05
test loss: 0.00010217682156938707
STEP: 2
loss: 9.73964008578994e-05
loss: 9.651190184335069e-05
loss: 9.426952857510346e-05
loss: 9.017719073172864e-05
loss: 8.286415009156239e-05
loss: 7.23559010019141e-05
loss: 6.0390239200254055e-05
loss: 5.0209257781695004e-05
loss: 4.2972552899404816e-05
loss: 3.980529133291303e-05
loss: 3.9761731526746026e-05

loss: 3.8483948024033276e-05
loss: 3.837196536310659e-05
loss: 3.831322707072425e-05
loss: 3.8234139582340796e-05
loss: 3.802988503873692e-05
loss: 3.7182075533095814e-05
loss: 3.581173166431694e-05
loss: 3.318974326607237e-05
loss: 2.9095838692330976e-05
test loss: 2.7529205522272456e-05
STEP: 3
loss: 2.400174289201227e-05
loss: 2.0072429844805332e-05
loss: 1.7845149275416788e-05
loss: 1.467782877040609e-05
loss: 1.4964701112867906e-05
loss: 1.3449303713546306e-05
loss: 1.2690239609178202e-05
loss: 1.2365039722026718e-05
loss: 1.225929541144198e-05
loss: 1.2176448126348446e-05
loss: 1.212221182681479e-05
loss: 1.209018906426645e-05
loss: 1.2066225141273267e-05
loss: 1.202675714453275e-05
loss: 1.1932072305013084e-05
loss: 1.1795063860501143e-05
loss: 1.1617440602322369e-05
loss: 1.1279644274258245e-05
loss: 1.0697804902486651e-05
loss: 1.0108701679774201e-05
test loss: 1.2199616332945017e-05
STEP: 4
loss: 9.429031026301022e-06
loss: 8.735457936425786e-06
loss: 8.315900675575274e-06
loss: 8.114112335609762e-06
loss: 8.017988442256067e-06
loss: 7.994299971184661e-06
loss: 7.987985571148376e-06
loss: 7.986290393240236e-06
loss: 7.950095764208726e-06
loss: 7.6248988618737836e-06
loss: 7.310652373434198e-06
loss: 6.990675500395126e-06
loss: 6.821272016336492e-06
loss: 6.793995656704354e-06
loss: 6.765836889332896e-06
loss: 6.762849350175792e-06
loss: 6.76053573527427e-06
loss: 6.757197404895279e-06
loss: 6.7523655134217975e-06
loss: 6.747100696112439e-06
test loss: 9.809934704825616e-06

Plot last result:

```
In [5]: #Show the last result here:
input = torch.from_numpy(data[3:, :-1])
plt.figure(figsize=(30,10))
plt.title('Predict future values for time sequences\n(Dashlines are predicted values)')
plt.xlabel('x', fontsize=20)
plt.ylabel('y', fontsize=20)
plt.xticks(fontsize=20)
plt.yticks(fontsize=20)
def draw(yi, color):
    plt.plot(np.arange(input.size(1)), yi[:input.size(1)], color, linewidth=2)
    plt.plot(np.arange(input.size(1), input.size(1) + future), yi[input.size(1):], color, linewidth=2)
draw(y[0], 'r')
draw(y[1], 'g')
draw(y[2], 'b')
plt.show()
```



Run the following to download the files from Google Colab

```
In [7]: from google.colab import files

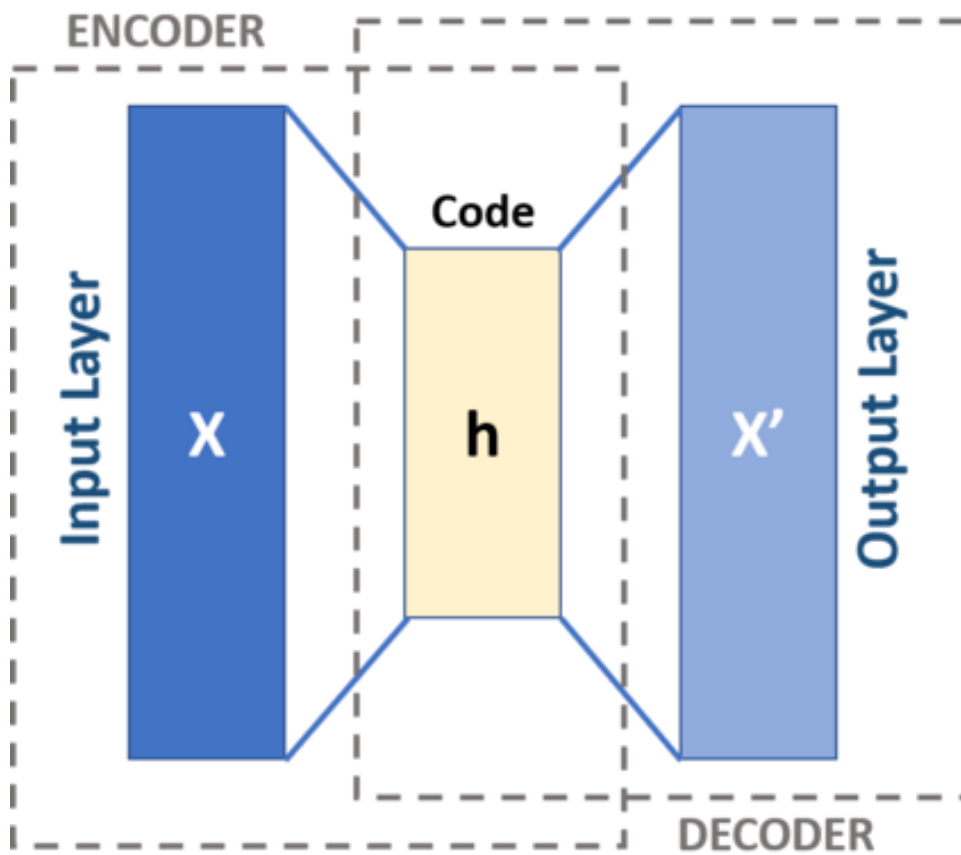
files_list = os.listdir()
for f in files_list:
    if ".pdf" in f:
        files.download(f)
```

Problem 2: Variational AutoEncoder Image Generation

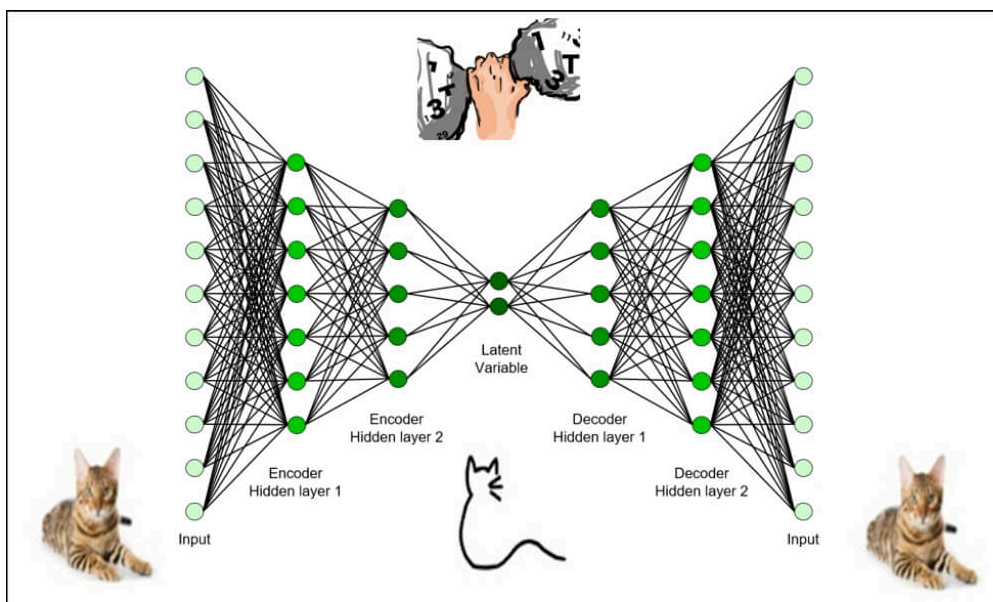
Autoencoders

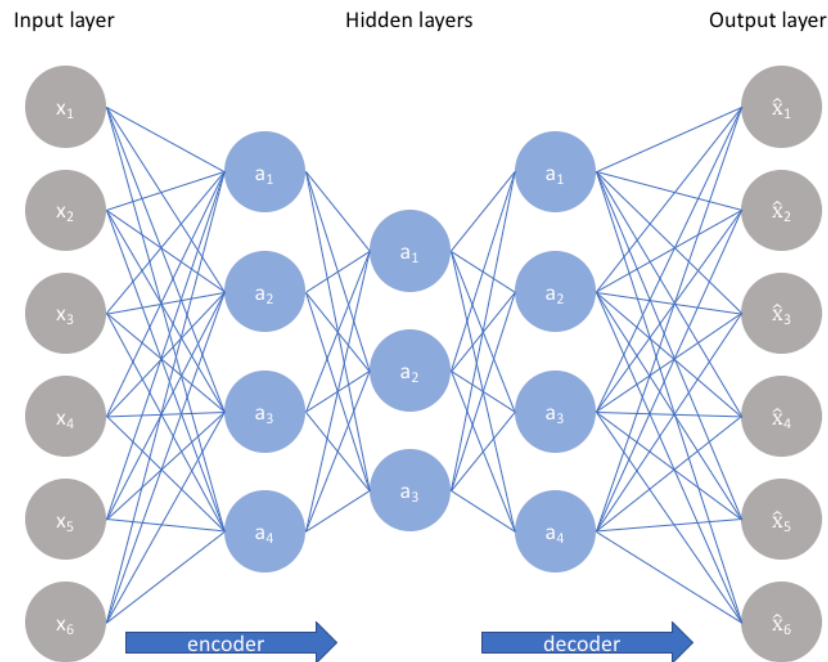
Autoencoders (AE) map a high-dimensional feature space to a compressed feature space (code h) then decompress to the output space of same dimension of the input space. The goal of compressing the feature space into the *code* is to make the representation of the key, essential information more compact. The component of the network that maps the high-dimensional input to the compressed code is termed the

Encoder, and the component of the network that decompresses the code to a representative high-dimensional feature space is termed the Decoder.

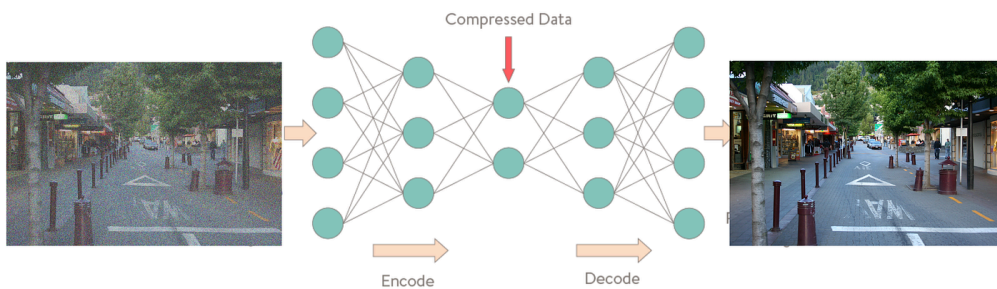


Now it is understood that due to compression, some information will inevitably be lost, but the essential information required to construct the essence of the input is retained in the code (h).





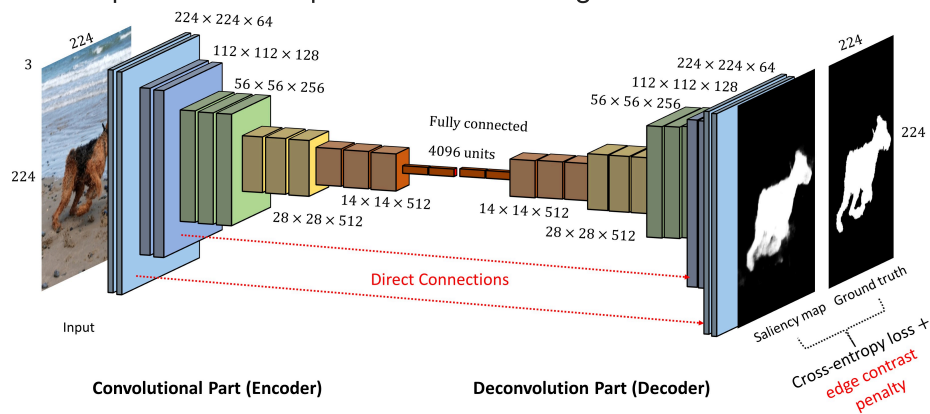
A practical example of this is the *denoising* of images:



which often requires the use of Convolutional Neural Networks (CNNs)

[example](#)

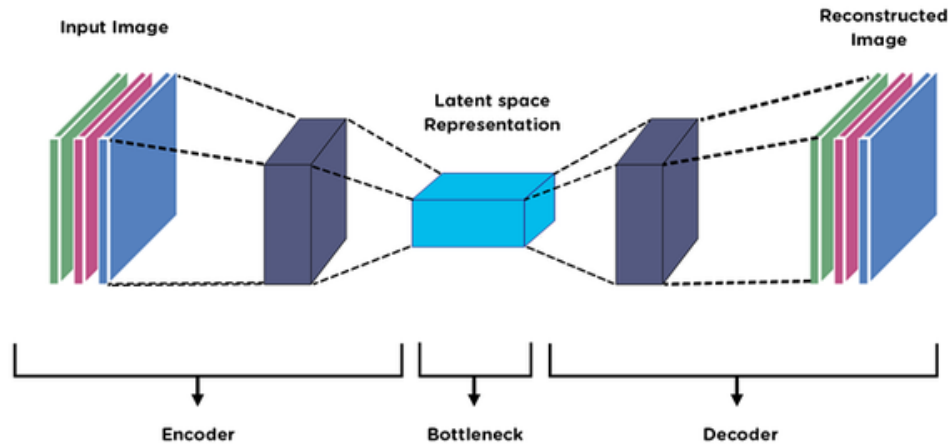
 No description has been provided for this image



But to perform supervised training of the Autoencoder, we must first define the loss function, which will be often be the Mean Squared Error (MSE) loss between the input x and the predicted output $\hat{x}$:

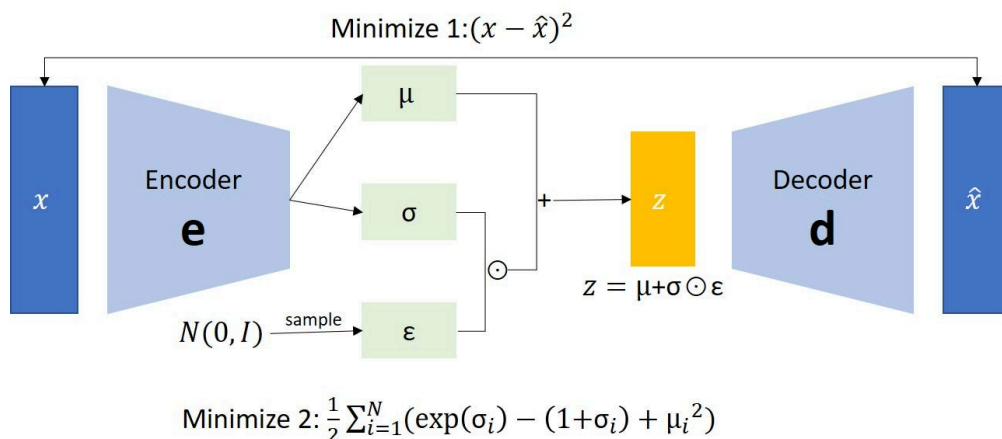
$$\mathcal{L}_{MSE} = ||x - \hat{x}||^2$$

of course this loss will never be zero, but by minimizing this loss, the prediction $\hat{x}$ will retain the salient features of the input x .



Variational Autoencoders

Variational Autoencoder (VAE) is an extension of the autoencoder that specifically accounts for the fact that *labeled* data might not have an injective (one-to-one) mapping, but may instead may be better represented by a *distribution* that is learned in the training data. That is to say, for a given input x it is not always best to predict an output $\hat{x}$ that is simply close to the given input x , but it is also important that for the given input, the output also reflect the distribution of similar points near x that was observed in the training data.



The key concept of the VAE comes from the use of Gaussian Mixture Models (GMMs) to approximate some (multivariate) distribution $P(x)$. To achieve this, a set of Gaussians is

required, and the goal is to at any point, select the appropriate Gaussian that best approximates the output of $P(x)$ at that point.

So in the autoencoder we had a code h , but now, we replace this code in the VAE with a continuous variable z which follows a standard normal distribution $N(0, 1)$ (0 mean, standard deviation of 1). So for each sample z there is an associated mean and standard deviation variables μ and σ of the Gaussian distribution corresponding to z . Just as in the GMM, if we integrate over all of the Gaussian distributions, the integrated domain becomes the original distribution $P(x)$:

$$P(x) = \int_z P(z)P(x|z)$$

Where we have that $z \sim N(0, 1)$ (z is sample from normal distribution with 0 mean and variance of 1), $x|z \sim N(\mu(z), \sigma(z))$ (estimated variable x is conditioned on the latent variable z is sampled from a Gaussian distribution whose mean and variance are related to the continuous latent variable z). At this point, $P(z)$ is known to be $(N(0, 1))$, and the conditional probability is what must be solved for as we need to find what μ, σ should be to help us approximate $P(x)$.

Our goal is to produce results that are very similar to the input x , which means our objective is to maximize our prediction $P(x)$ (likelihood that we predict a value very similar to the input). If this is achieved on training points, then it is likely to produce similar samples when deployed.

So to maximize the $P(x)$, we can state this objective as L :

$$\max L = \sum_x \log(P(x))$$

Here we leverage the log of the likelihood ($P(x)$) to give us a nice monotonically increasing function related to probability ($P(x) \in [0, 1]$, therefore $\log(P(x)) \in [-\infty, 0]$ where when maximized $\log(P(x)) = \log(1) = 0$).

Given the objective to maximize $\log(P(x))$, we can first expand this term as follows:

We introduce the distribution $q(z|x)$ which once summed over z produces $\log P(x)$:

$$\log P(x) = \int_z q(z|x) \log(P(x)) dz$$

We then use [Bayes Theorem](#) for dependent variables to make the substitution in the $\log(\cdot)$ argument:

$$= \int_z q(z|x) \log\left(\frac{P(z, x)}{P(z|x)}\right) dz$$

We then multiply the $\log(\cdot)$ argument by 1, introducing to the numerator and denominator the distribution $q(z|x)$:

$$= \int_z q(z|x) \log \left(\frac{P(z, x)q(z|x)}{q(z|x)P(z|x)} \right) dz$$

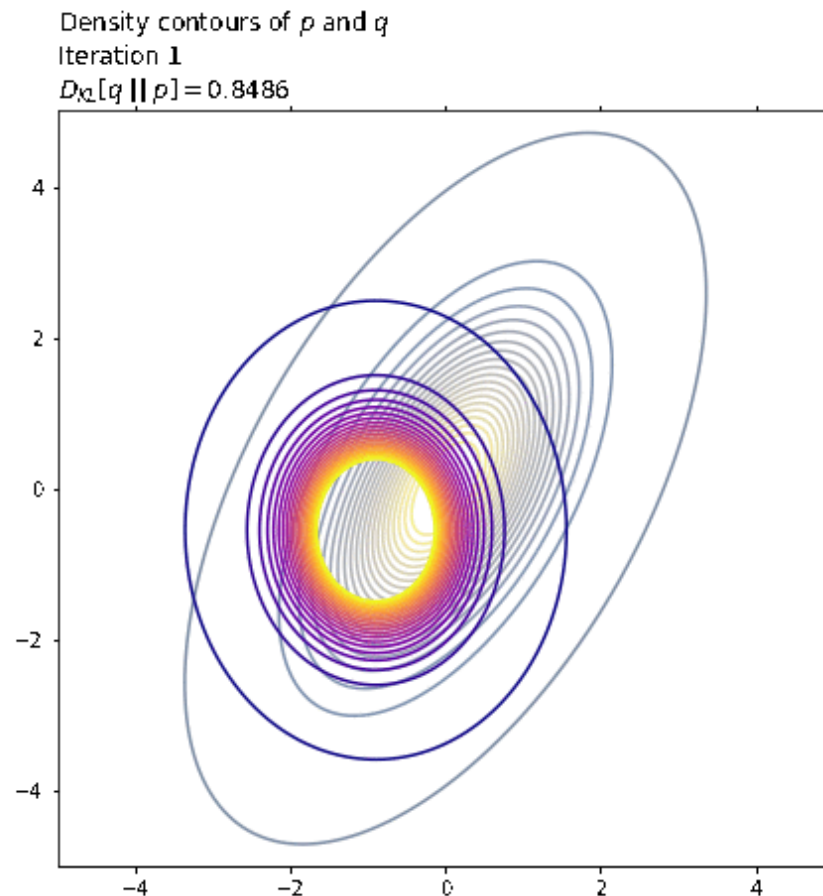
By [logarithmic identities](#) we can turn this into two integrals separating terms as follows:

$$= \int_z q(z|x) \log \left(\frac{P(z, x)}{q(z|x)} \right) dz + \int_z q(z|x) \log \left(\frac{q(z|x)}{P(z|x)} \right) dz$$

The second term is the [Kullback–Leibler \(KL\) divergence](#) which has a value greater than or equal to zero:

$$= \int_z q(z|x) \log \left(\frac{P(z, x)}{q(z|x)} \right) dz + D_{KL}(q(z|x) || P(z|x))$$

The KL divergence (also called relative entropy) is a statistical distance of how different two probability distributions are from each other, but the ordering matters for this metric. For $D_{KL}(Q || P)$ (the divergence of Q from P) is the expected excess *surprise* from using P as a model when the actual distribution is Q.



[another great animation of KL divergence]

(https://twitter.com/ari_seff/status/1303741288911638530)

Because the KL term is equal to or greater than 0, we've found a lower bound of the $\log P(x)$ (again also leveraging Bayes theorem):

$$\log P(x) \geq \int_z q(z|x) \log \left(\frac{P(x|z)P(z)}{q(z|x)} \right) dz$$

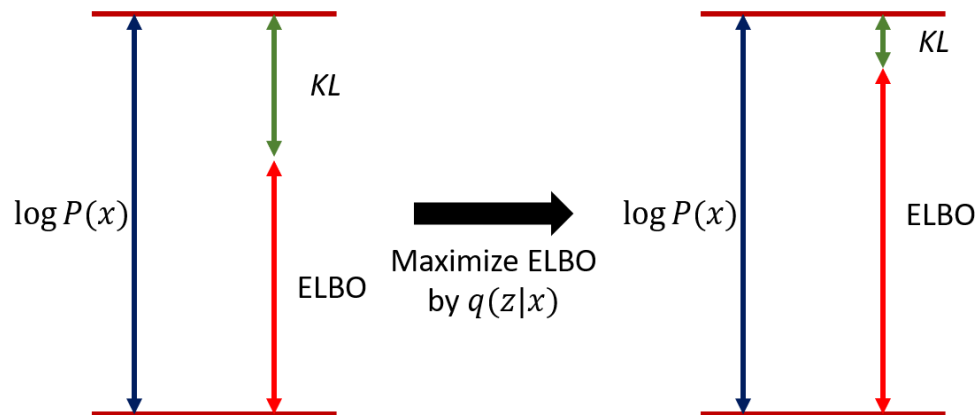
We call this the Evidence Based Lower Bound (ELBO):

$$\log P(x) \geq ELBO = \int_z q(z|x) \log \left(\frac{P(x|z)P(z)}{q(z|x)} \right) dz = E_{q(z|x)} \left[\log \frac{p(x, z)}{q(z|x)} \right]$$

The last term on the right above leverages **Expected value** notation $E[\cdot]$.

So we can rewrite the original form as follows:

$$\log P(x) = ELBO + D_{KL}(q(z|x) || P(z|x))$$



By adjusting $q(z|x)$ to make the ELBO higher, the KL divergence must get smaller and smaller. We continue to update $q(z|x)$ until it is the same as $P(z|x)$ making the KL divergence converge to 0, and the ELBO and $\log P(x)$ are fully consistent. Maximizing $\log P(x)$ is equivalent to maximizing the ELBO.

Adjusting $P(x|z)$ is achieved by adjusting the Decoder, and adjusting $q(z|x)$ is adjusting the encoder. So everytime the decoder is updated, the encoder must be adjusted to be consistent with it so that the decoder will be continue to improve after each next training epoch.

We can further simplify our ELBO expression with the following derivation:

$$ELBO = \int_z q(z|x) \log \left(\frac{P(x|z)P(z)}{q(z|x)} \right) dz$$

by logarithmic identities ($\log(ab) = \log(a) + \log(b)$) where here $a = \frac{P(z)}{q(z|x)}$, $b = P(x|z)$

:

$$ELBO = \int_z q(z|x) \log\left(\frac{P(z)}{q(z|x)}\right) dz + \int_z q(z|x) \log(P(x|z)) dz$$

This can be rewritten using [KL divergence identity](#)

$$ELBO = -D_{KL}(q(z|x)||P(z)) + \int_z q(z|x) \log(P(x|z)) dz$$

Therefore, maximizing the ELBO is equivalent to minimize $D_{KL}(q(z|x)||P(z))$ (as it is a negative value) and maximizing the integral equation in the second term of the above:

$$\max \int_z q(z|x) \log(P(x|z)) dz = \max E_{q(z|x)}[\log P(x|z)]$$

The above expectation $\max E_{q(z|x)}[\log P(x|z)]$ means maximize the likelihood $P(x|z)$ (Decoder's output) given that $q(z|x)$ (Encoder's output) is as high as possible. This is similar to AutoEncoder's loss function (reconstruction error):

$$L = (x - \hat{x})^2$$

Finally, we can simplify $-D_{KL}(q(z|x)||P(z))$ by first decomposing its components using the logarithmic properties $\log(\frac{a}{b}) = \log(a) - \log(b)$ where $a = P(z)$ and $b = q(z|x)$, so for $P(z)$:

$$\int_z q(z|x) \log P(z) dz = \int_z N(z; \mu, \sigma^2) \log N(z; 0, I) dz$$

which for the special case of our distributions being Gaussian, is equivalent to

$$= -\frac{J}{2} \log(2\pi) - \frac{1}{2} \sum_{i=1}^J (\mu_i^2 + \sigma_i^2)$$

where J is the dimensionality of z , μ, σ denote the variational mean and standard deviation vectors, and μ_i, σ_i simply denote the i -th element of these vectors.

Likewise, for $q(z|x)$:

$$\int_z q(z|x) \log q(z|x) dz = \int_z N(z; \mu, \sigma^2) \log N(z; \mu, \sigma^2) dz$$

which for the Gaussian case is equivalent to the closed form:

$$= -\frac{J}{2} \log(2\pi) - \frac{1}{2} \sum_{i=1}^J (1 + \log \sigma_i^2)$$

Therefore we can write $-KL(q(z|x)||P(z))$ as:

$$-D_{KL}(q(z|x)||P(z)) = \int_z q(z|x) (\log P(z) - \log q(z|x)) dz$$

which in closed form is:

$$-D_{KL}(q(z|x)||P(z)) = -\frac{1}{2} \sum_{i=1}^J (1 + \log(\sigma_i^2) - \mu_i^2 - \sigma_i^2)$$

where again, μ_i, σ_i simply denote the i-th element of these vectors μ, σ .

If we use a Bernoulli or Gaussian Multi-Layer Perception (MLP) to capture the distributions $q(z|x) = q_\phi(z|x)$, $P(z) = p_\theta(z)$, $P(x|z) = p_\theta(x|z)$, then for a marginal likelihood composed of a sum over the marginal likelihoods of individual datapoints $\log p_\theta(x^{(1)}, \dots, x^{(N)}) = \sum_{i=1}^N \log p_\theta(x^{(i)})$ then this marginal likelihood we've seen for the i-th datapoint is:

$$\log p_\theta(x^{(i)}) = \underbrace{D_{KL}(q_\phi(z|x^{(i)})||p_\theta(z|x^{(i)}))}_{\text{regularization}} + \underbrace{\mathcal{L}(\theta, \phi; x^{(i)})}_{\text{loss}}$$

Then the loss term is:

$$L(\theta, \phi; x^{(i)}) = \underbrace{-D_{KL}(q_\phi(z|x^{(i)})||p_\theta(z))}_{\text{regularization}} + \underbrace{\mathbb{E}_{q_\phi(z|x^{(i)})} [\log p_\theta(x^{(i)}|z)]}_{\text{reconstruction}}$$

Which simplified becomes:

$$L(\theta, \phi; x^{(i)}) \simeq \frac{1}{2} \sum_{j=1}^J \left(1 + \log((\sigma_j^{(i)})^2) - (\mu_j^{(i)})^2 - (\sigma_j^{(i)})^2 \right) + \frac{1}{L} \sum_{l=1}^L \log p_\theta(x^{(i)}|z^{(i,l)})$$

where from the *reparameterization trick* we have $z^{(i,l)} = \mu^{(i)} + \underbrace{\sigma^{(i)} \odot \epsilon^{(l)}}_{\text{element-wise product}}$ and

$$\epsilon^{(l)} \sim \mathcal{N}(0, I)$$

Good References:

- [Auto-Encoding Variational Bayes](#)
- [Variational Autoencoder \(VAE\) - Roger](#)
- [Tutorial on Variational Autoencoders](#)

Your Problem: MNIST Dataset

(note: this problem requires internet access to download the dataset)

In this problem you will leverage a VAE to perform image reconstruction for a hand-written number given an input. Your output will be in distribution of the original input (an

auto-generated 'hand-written' number of the same type)



Your Code:

(see spots indicated "fill this in")

```
In [11]: def student_function_2(train_loader, test_loader, batch_size = 128, epochs = 10000)
        """
        Description: This is an implementation of a VAE for written text generation. The
        network is capable of producing another written number that looks very similar to the
        number it was shown.

        Parameters
        -----
        train_loader : torch.utils.data.DataLoader
            object for loading training data
        test_loader : torch.utils.data.DataLoader
            Object for loading test data
        cuda_bool : bool
            boolean if GPU (cuda) is available
        batch_size : int
            size of each batch for a given epoch
        epochs : int
            number of epochs for training
        log_interval : int
            This is a counter to display training progress

        device : string
            does this machine have GPU (cuda) or just CPU? (default is CPU, but
            Returns
            -----
        implement_bool : bool
            true/false - has the function been implemented?
```

```

#####
# raise NotImplementedError #comment/delete this once implemented

class VAE(nn.Module):
    def __init__(self):
        super(VAE, self).__init__()
        #Define the architecture
        self.fc1 = nn.Linear(784, 400)
        self.fc21 = nn.Linear(400, 20)
        self.fc22 = nn.Linear(400, 20)
        self.fc3 = nn.Linear(20, 400)
        self.fc4 = nn.Linear(400, 784)

    def encode(self, x):
        h1 = F.relu(self.fc1(x))
        return self.fc21(h1), self.fc22(h1)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5*logvar)
        eps = torch.randn_like(std)
        return mu + eps*std

    def decode(self, z):
        h3 = F.relu(self.fc3(z))
        return torch.sigmoid(self.fc4(h3))

    def forward(self, x):
        mu, logvar = self.encode(x.view(-1, 784))
        z = self.reparameterize(mu, logvar)
        return self.decode(z), mu, logvar

model = VAE().to(device)
optimizer = optim.Adam(model.parameters(), lr=1e-3)

# Reconstruction + KL divergence losses summed over all elements and batches
def loss_function(recon_x, x, mu, logvar):
    # Reconstruction loss (classified correctly in this problem, could be improved)
    # Binary cross entropy:
    BCE = F.binary_cross_entropy(recon_x, x.view(-1, 784), reduction='sum')

    # see Appendix B from VAE paper:
    # Kingma and Welling. Auto-Encoding Variational Bayes. ICLR, 2014
    # https://arxiv.org/abs/1312.6114
    #  $KLD = -0.5 * \sum(1 + \log(\sigma^2) - \mu^2 - \sigma^2)$ 
    # Hint: use torch.sum(...) for summation; mu.pow(2) for  $\mu^2$ ; logvar.exp() for  $\sigma^2$ 

    #FILL THIS IN
    KLD = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())

    return BCE + KLD

def train(epoch):
    model.train()

```

```

train_loss = 0
for batch_idx, (data, _) in enumerate(train_loader):
    data = data.to(device)
    #Zero the optimizer gradients (for this epoch)
    optimizer.zero_grad()
    #get model output from input
    recon_batch, mu, logvar = model(data)
    #Get the loss:
    loss = loss_function(recon_batch, data, mu, logvar) #Make sure t
    #Perform backpropogation to adjust weights for better prediction
    loss.backward()
    train_loss += loss.item()
    #update the model:
    optimizer.step()
    if batch_idx % log_interval == 0:
        print('Train Epoch: {} [{}/{} ( {:.0f}% )]\tLoss: {:.6f}'.format(
            epoch, batch_idx * len(data), len(train_loader.dataset),
            100. * batch_idx / len(train_loader),
            loss.item() / len(data)))

print('====> Epoch: {} Average loss: {:.4f}'.format(
    epoch, train_loss / len(train_loader.dataset)))

def test(epoch):
    model.eval() #put model in evaluation mode
    test_loss = 0
    with torch.no_grad():
        for i, (data, _) in enumerate(test_loader):
            data = data.to(device) #Use the device you have (cpu vs gpu)
            recon_batch, mu, logvar = model(data) #evaluate the model
            test_loss += loss_function(recon_batch, data, mu, logvar).item()
            #save images from performance:
            if i == 0:
                n = min(data.size(0), 8)
                comparison = torch.cat([data[:n],
                                         recon_batch.view(batch_size, 1, 28, 28)],
                                         dim=1)
                save_image(comparison.cpu(),
                           'results/reconstruction_' + str(epoch) + '.png')

    test_loss /= len(test_loader.dataset)
    print('====> Test set loss: {:.4f}'.format(test_loss))

#Here is where things start in this script:
for epoch in range(1, epochs + 1):
    train(epoch) #train for the epoch
    test(epoch) #test for epoch

    with torch.no_grad():
        #display current results
        sample = torch.randn(64, 20).to(device)
        sample = model.decode(sample).cpu()
        save_image(sample.view(64, 1, 28, 28),
                    'results/sample_' + str(epoch) + '.png') #this sample

```



```
implement_bool = True #has script been implemented?
return implement_bool
```

Evaluation

```
In [14]: #Setup Necessary variables"

#make a 'results' folder for putting results:
MYDIR = ("results")
CHECK_FOLDER = os.path.isdir(MYDIR)
if not CHECK_FOLDER: os.makedirs(MYDIR) # If folder doesn't exist, then crea

#Set Arguments
cuda_bool = torch.cuda.is_available() #Determine if GPU is available (cuda)
batch_size = 128 #size of batches (how many imgs to train together)
epochs = 10 #iterations of convergence
log_interval = 10 #count for printing training progress
seed = 1 #value for seed values for network weights
torch.manual_seed(seed) #seed value for network weights
#use GPU if available:
device = torch.device("cuda" if cuda_bool else "cpu")

#Load data:
kwargs = {'num_workers': 1, 'pin_memory': True} if cuda_bool else {}
train_loader = torch.utils.data.DataLoader(
    datasets.MNIST(data_path, train=True, download=True,
                   transform=transforms.ToTensor()),
    batch_size=batch_size, shuffle=True, **kwargs)
test_loader = torch.utils.data.DataLoader(
    datasets.MNIST(data_path, train=False, transform=transforms.ToTensor()),
    batch_size=batch_size, shuffle=True, **kwargs)

#Your code:
complete_bool = student_function_2(train_loader, test_loader, device=device)
```

Train Epoch: 1	[0/60000 (0%)]	Loss: 550.513916
Train Epoch: 1	[1280/60000 (2%)]	Loss: 310.610535
Train Epoch: 1	[2560/60000 (4%)]	Loss: 240.650436
Train Epoch: 1	[3840/60000 (6%)]	Loss: 219.223083
Train Epoch: 1	[5120/60000 (9%)]	Loss: 215.092896
Train Epoch: 1	[6400/60000 (11%)]	Loss: 208.449524
Train Epoch: 1	[7680/60000 (13%)]	Loss: 203.374680
Train Epoch: 1	[8960/60000 (15%)]	Loss: 193.757126
Train Epoch: 1	[10240/60000 (17%)]	Loss: 194.697510
Train Epoch: 1	[11520/60000 (19%)]	Loss: 192.934937
Train Epoch: 1	[12800/60000 (21%)]	Loss: 179.947601
Train Epoch: 1	[14080/60000 (23%)]	Loss: 174.070770
Train Epoch: 1	[15360/60000 (26%)]	Loss: 182.592896
Train Epoch: 1	[16640/60000 (28%)]	Loss: 168.164948
Train Epoch: 1	[17920/60000 (30%)]	Loss: 166.453857
Train Epoch: 1	[19200/60000 (32%)]	Loss: 161.456146
Train Epoch: 1	[20480/60000 (34%)]	Loss: 162.513458
Train Epoch: 1	[21760/60000 (36%)]	Loss: 151.486923
Train Epoch: 1	[23040/60000 (38%)]	Loss: 157.244186
Train Epoch: 1	[24320/60000 (41%)]	Loss: 154.493698
Train Epoch: 1	[25600/60000 (43%)]	Loss: 155.926086
Train Epoch: 1	[26880/60000 (45%)]	Loss: 150.694138
Train Epoch: 1	[28160/60000 (47%)]	Loss: 153.045181
Train Epoch: 1	[29440/60000 (49%)]	Loss: 146.293472
Train Epoch: 1	[30720/60000 (51%)]	Loss: 141.652695
Train Epoch: 1	[32000/60000 (53%)]	Loss: 144.752121
Train Epoch: 1	[33280/60000 (55%)]	Loss: 144.582489
Train Epoch: 1	[34560/60000 (58%)]	Loss: 145.722214
Train Epoch: 1	[35840/60000 (60%)]	Loss: 137.231903
Train Epoch: 1	[37120/60000 (62%)]	Loss: 142.204163
Train Epoch: 1	[38400/60000 (64%)]	Loss: 139.166794
Train Epoch: 1	[39680/60000 (66%)]	Loss: 135.014938
Train Epoch: 1	[40960/60000 (68%)]	Loss: 141.468842
Train Epoch: 1	[42240/60000 (70%)]	Loss: 141.841888
Train Epoch: 1	[43520/60000 (72%)]	Loss: 140.196793
Train Epoch: 1	[44800/60000 (75%)]	Loss: 135.171707
Train Epoch: 1	[46080/60000 (77%)]	Loss: 140.352570
Train Epoch: 1	[47360/60000 (79%)]	Loss: 132.946930
Train Epoch: 1	[48640/60000 (81%)]	Loss: 131.527695
Train Epoch: 1	[49920/60000 (83%)]	Loss: 130.478806
Train Epoch: 1	[51200/60000 (85%)]	Loss: 131.258514
Train Epoch: 1	[52480/60000 (87%)]	Loss: 131.104935
Train Epoch: 1	[53760/60000 (90%)]	Loss: 132.594467
Train Epoch: 1	[55040/60000 (92%)]	Loss: 126.495804
Train Epoch: 1	[56320/60000 (94%)]	Loss: 137.629684
Train Epoch: 1	[57600/60000 (96%)]	Loss: 129.705765
Train Epoch: 1	[58880/60000 (98%)]	Loss: 132.282867
====>	Epoch: 1 Average loss:	164.3825
====>	Test set loss:	127.1136
Train Epoch: 2	[0/60000 (0%)]	Loss: 125.500549
Train Epoch: 2	[1280/60000 (2%)]	Loss: 131.346313
Train Epoch: 2	[2560/60000 (4%)]	Loss: 128.355774
Train Epoch: 2	[3840/60000 (6%)]	Loss: 129.393326
Train Epoch: 2	[5120/60000 (9%)]	Loss: 127.003510
Train Epoch: 2	[6400/60000 (11%)]	Loss: 124.429779
Train Epoch: 2	[7680/60000 (13%)]	Loss: 125.548294

Train Epoch: 2	[8960/60000 (15%)]	Loss: 131.641556
Train Epoch: 2	[10240/60000 (17%)]	Loss: 124.382080
Train Epoch: 2	[11520/60000 (19%)]	Loss: 129.440231
Train Epoch: 2	[12800/60000 (21%)]	Loss: 124.941551
Train Epoch: 2	[14080/60000 (23%)]	Loss: 123.889160
Train Epoch: 2	[15360/60000 (26%)]	Loss: 129.186356
Train Epoch: 2	[16640/60000 (28%)]	Loss: 117.873016
Train Epoch: 2	[17920/60000 (30%)]	Loss: 128.668015
Train Epoch: 2	[19200/60000 (32%)]	Loss: 126.656631
Train Epoch: 2	[20480/60000 (34%)]	Loss: 122.001297
Train Epoch: 2	[21760/60000 (36%)]	Loss: 121.975555
Train Epoch: 2	[23040/60000 (38%)]	Loss: 123.333733
Train Epoch: 2	[24320/60000 (41%)]	Loss: 126.267273
Train Epoch: 2	[25600/60000 (43%)]	Loss: 123.385223
Train Epoch: 2	[26880/60000 (45%)]	Loss: 122.751801
Train Epoch: 2	[28160/60000 (47%)]	Loss: 120.790756
Train Epoch: 2	[29440/60000 (49%)]	Loss: 122.014145
Train Epoch: 2	[30720/60000 (51%)]	Loss: 121.319252
Train Epoch: 2	[32000/60000 (53%)]	Loss: 117.558449
Train Epoch: 2	[33280/60000 (55%)]	Loss: 114.850662
Train Epoch: 2	[34560/60000 (58%)]	Loss: 128.626266
Train Epoch: 2	[35840/60000 (60%)]	Loss: 118.609512
Train Epoch: 2	[37120/60000 (62%)]	Loss: 118.949707
Train Epoch: 2	[38400/60000 (64%)]	Loss: 118.364822
Train Epoch: 2	[39680/60000 (66%)]	Loss: 118.060242
Train Epoch: 2	[40960/60000 (68%)]	Loss: 118.401764
Train Epoch: 2	[42240/60000 (70%)]	Loss: 118.658356
Train Epoch: 2	[43520/60000 (72%)]	Loss: 116.417908
Train Epoch: 2	[44800/60000 (75%)]	Loss: 114.761749
Train Epoch: 2	[46080/60000 (77%)]	Loss: 117.126328
Train Epoch: 2	[47360/60000 (79%)]	Loss: 118.418694
Train Epoch: 2	[48640/60000 (81%)]	Loss: 119.066368
Train Epoch: 2	[49920/60000 (83%)]	Loss: 123.089165
Train Epoch: 2	[51200/60000 (85%)]	Loss: 117.248024
Train Epoch: 2	[52480/60000 (87%)]	Loss: 115.504066
Train Epoch: 2	[53760/60000 (90%)]	Loss: 119.260834
Train Epoch: 2	[55040/60000 (92%)]	Loss: 116.385536
Train Epoch: 2	[56320/60000 (94%)]	Loss: 116.976364
Train Epoch: 2	[57600/60000 (96%)]	Loss: 119.733498
Train Epoch: 2	[58880/60000 (98%)]	Loss: 116.634743
====>	Epoch: 2 Average loss: 121.5007	
====>	Test set loss: 116.0667	
Train Epoch: 3	[0/60000 (0%)]	Loss: 120.293556
Train Epoch: 3	[1280/60000 (2%)]	Loss: 120.155807
Train Epoch: 3	[2560/60000 (4%)]	Loss: 114.545067
Train Epoch: 3	[3840/60000 (6%)]	Loss: 114.624397
Train Epoch: 3	[5120/60000 (9%)]	Loss: 117.016479
Train Epoch: 3	[6400/60000 (11%)]	Loss: 110.463150
Train Epoch: 3	[7680/60000 (13%)]	Loss: 116.509705
Train Epoch: 3	[8960/60000 (15%)]	Loss: 114.858978
Train Epoch: 3	[10240/60000 (17%)]	Loss: 116.231674
Train Epoch: 3	[11520/60000 (19%)]	Loss: 113.391029
Train Epoch: 3	[12800/60000 (21%)]	Loss: 113.244209
Train Epoch: 3	[14080/60000 (23%)]	Loss: 116.027794
Train Epoch: 3	[15360/60000 (26%)]	Loss: 112.971527
Train Epoch: 3	[16640/60000 (28%)]	Loss: 116.521858

Train Epoch: 3	[17920/60000 (30%)]	Loss: 112.031189
Train Epoch: 3	[19200/60000 (32%)]	Loss: 115.842926
Train Epoch: 3	[20480/60000 (34%)]	Loss: 118.746201
Train Epoch: 3	[21760/60000 (36%)]	Loss: 117.815933
Train Epoch: 3	[23040/60000 (38%)]	Loss: 113.994728
Train Epoch: 3	[24320/60000 (41%)]	Loss: 120.016541
Train Epoch: 3	[25600/60000 (43%)]	Loss: 114.663757
Train Epoch: 3	[26880/60000 (45%)]	Loss: 115.680939
Train Epoch: 3	[28160/60000 (47%)]	Loss: 114.510803
Train Epoch: 3	[29440/60000 (49%)]	Loss: 115.007446
Train Epoch: 3	[30720/60000 (51%)]	Loss: 118.457016
Train Epoch: 3	[32000/60000 (53%)]	Loss: 114.646721
Train Epoch: 3	[33280/60000 (55%)]	Loss: 115.048317
Train Epoch: 3	[34560/60000 (58%)]	Loss: 115.422813
Train Epoch: 3	[35840/60000 (60%)]	Loss: 116.295135
Train Epoch: 3	[37120/60000 (62%)]	Loss: 116.230423
Train Epoch: 3	[38400/60000 (64%)]	Loss: 115.120583
Train Epoch: 3	[39680/60000 (66%)]	Loss: 115.567841
Train Epoch: 3	[40960/60000 (68%)]	Loss: 116.723862
Train Epoch: 3	[42240/60000 (70%)]	Loss: 110.980164
Train Epoch: 3	[43520/60000 (72%)]	Loss: 111.994781
Train Epoch: 3	[44800/60000 (75%)]	Loss: 111.190071
Train Epoch: 3	[46080/60000 (77%)]	Loss: 112.925751
Train Epoch: 3	[47360/60000 (79%)]	Loss: 117.541077
Train Epoch: 3	[48640/60000 (81%)]	Loss: 114.218933
Train Epoch: 3	[49920/60000 (83%)]	Loss: 113.971817
Train Epoch: 3	[51200/60000 (85%)]	Loss: 112.850128
Train Epoch: 3	[52480/60000 (87%)]	Loss: 113.273346
Train Epoch: 3	[53760/60000 (90%)]	Loss: 111.112396
Train Epoch: 3	[55040/60000 (92%)]	Loss: 115.475998
Train Epoch: 3	[56320/60000 (94%)]	Loss: 112.461548
Train Epoch: 3	[57600/60000 (96%)]	Loss: 114.929146
Train Epoch: 3	[58880/60000 (98%)]	Loss: 109.808701
====>	Epoch: 3 Average loss: 114.4030	
====>	Test set loss: 111.6667	
Train Epoch: 4	[0/60000 (0%)]	Loss: 113.827423
Train Epoch: 4	[1280/60000 (2%)]	Loss: 117.254997
Train Epoch: 4	[2560/60000 (4%)]	Loss: 114.631355
Train Epoch: 4	[3840/60000 (6%)]	Loss: 112.580872
Train Epoch: 4	[5120/60000 (9%)]	Loss: 114.517395
Train Epoch: 4	[6400/60000 (11%)]	Loss: 113.299446
Train Epoch: 4	[7680/60000 (13%)]	Loss: 113.282028
Train Epoch: 4	[8960/60000 (15%)]	Loss: 112.330910
Train Epoch: 4	[10240/60000 (17%)]	Loss: 111.497910
Train Epoch: 4	[11520/60000 (19%)]	Loss: 108.304245
Train Epoch: 4	[12800/60000 (21%)]	Loss: 112.643692
Train Epoch: 4	[14080/60000 (23%)]	Loss: 111.060135
Train Epoch: 4	[15360/60000 (26%)]	Loss: 113.022667
Train Epoch: 4	[16640/60000 (28%)]	Loss: 109.261612
Train Epoch: 4	[17920/60000 (30%)]	Loss: 112.215225
Train Epoch: 4	[19200/60000 (32%)]	Loss: 106.986847
Train Epoch: 4	[20480/60000 (34%)]	Loss: 106.842468
Train Epoch: 4	[21760/60000 (36%)]	Loss: 112.621094
Train Epoch: 4	[23040/60000 (38%)]	Loss: 110.909027
Train Epoch: 4	[24320/60000 (41%)]	Loss: 110.886200
Train Epoch: 4	[25600/60000 (43%)]	Loss: 114.574120

Train Epoch: 4	[26880/60000 (45%)]	Loss: 115.399078
Train Epoch: 4	[28160/60000 (47%)]	Loss: 111.937889
Train Epoch: 4	[29440/60000 (49%)]	Loss: 111.150932
Train Epoch: 4	[30720/60000 (51%)]	Loss: 108.950218
Train Epoch: 4	[32000/60000 (53%)]	Loss: 110.315186
Train Epoch: 4	[33280/60000 (55%)]	Loss: 107.973709
Train Epoch: 4	[34560/60000 (58%)]	Loss: 109.621529
Train Epoch: 4	[35840/60000 (60%)]	Loss: 106.205750
Train Epoch: 4	[37120/60000 (62%)]	Loss: 110.335342
Train Epoch: 4	[38400/60000 (64%)]	Loss: 112.008682
Train Epoch: 4	[39680/60000 (66%)]	Loss: 114.714050
Train Epoch: 4	[40960/60000 (68%)]	Loss: 112.635696
Train Epoch: 4	[42240/60000 (70%)]	Loss: 108.802246
Train Epoch: 4	[43520/60000 (72%)]	Loss: 113.483414
Train Epoch: 4	[44800/60000 (75%)]	Loss: 106.423645
Train Epoch: 4	[46080/60000 (77%)]	Loss: 110.463966
Train Epoch: 4	[47360/60000 (79%)]	Loss: 113.644989
Train Epoch: 4	[48640/60000 (81%)]	Loss: 110.414642
Train Epoch: 4	[49920/60000 (83%)]	Loss: 114.506813
Train Epoch: 4	[51200/60000 (85%)]	Loss: 112.709694
Train Epoch: 4	[52480/60000 (87%)]	Loss: 112.287720
Train Epoch: 4	[53760/60000 (90%)]	Loss: 110.149513
Train Epoch: 4	[55040/60000 (92%)]	Loss: 109.464577
Train Epoch: 4	[56320/60000 (94%)]	Loss: 111.707771
Train Epoch: 4	[57600/60000 (96%)]	Loss: 113.515930
Train Epoch: 4	[58880/60000 (98%)]	Loss: 111.716301
====>	Epoch: 4 Average loss: 111.4705	
====>	Test set loss: 109.8224	
Train Epoch: 5	[0/60000 (0%)]	Loss: 112.918198
Train Epoch: 5	[1280/60000 (2%)]	Loss: 109.150124
Train Epoch: 5	[2560/60000 (4%)]	Loss: 108.020386
Train Epoch: 5	[3840/60000 (6%)]	Loss: 108.191925
Train Epoch: 5	[5120/60000 (9%)]	Loss: 109.557953
Train Epoch: 5	[6400/60000 (11%)]	Loss: 111.507973
Train Epoch: 5	[7680/60000 (13%)]	Loss: 112.321014
Train Epoch: 5	[8960/60000 (15%)]	Loss: 111.955482
Train Epoch: 5	[10240/60000 (17%)]	Loss: 106.811646
Train Epoch: 5	[11520/60000 (19%)]	Loss: 109.552780
Train Epoch: 5	[12800/60000 (21%)]	Loss: 110.986877
Train Epoch: 5	[14080/60000 (23%)]	Loss: 106.328117
Train Epoch: 5	[15360/60000 (26%)]	Loss: 108.487366
Train Epoch: 5	[16640/60000 (28%)]	Loss: 108.271324
Train Epoch: 5	[17920/60000 (30%)]	Loss: 104.990593
Train Epoch: 5	[19200/60000 (32%)]	Loss: 108.760956
Train Epoch: 5	[20480/60000 (34%)]	Loss: 108.793823
Train Epoch: 5	[21760/60000 (36%)]	Loss: 111.697159
Train Epoch: 5	[23040/60000 (38%)]	Loss: 114.052200
Train Epoch: 5	[24320/60000 (41%)]	Loss: 108.426636
Train Epoch: 5	[25600/60000 (43%)]	Loss: 110.433098
Train Epoch: 5	[26880/60000 (45%)]	Loss: 109.178337
Train Epoch: 5	[28160/60000 (47%)]	Loss: 104.993950
Train Epoch: 5	[29440/60000 (49%)]	Loss: 111.009758
Train Epoch: 5	[30720/60000 (51%)]	Loss: 110.396545
Train Epoch: 5	[32000/60000 (53%)]	Loss: 109.124916
Train Epoch: 5	[33280/60000 (55%)]	Loss: 112.459167
Train Epoch: 5	[34560/60000 (58%)]	Loss: 109.597778

Train Epoch: 5	[35840/60000 (60%)]	Loss: 106.670578
Train Epoch: 5	[37120/60000 (62%)]	Loss: 109.416763
Train Epoch: 5	[38400/60000 (64%)]	Loss: 109.956390
Train Epoch: 5	[39680/60000 (66%)]	Loss: 105.868004
Train Epoch: 5	[40960/60000 (68%)]	Loss: 111.044151
Train Epoch: 5	[42240/60000 (70%)]	Loss: 105.680832
Train Epoch: 5	[43520/60000 (72%)]	Loss: 113.694458
Train Epoch: 5	[44800/60000 (75%)]	Loss: 107.480545
Train Epoch: 5	[46080/60000 (77%)]	Loss: 108.992485
Train Epoch: 5	[47360/60000 (79%)]	Loss: 108.111282
Train Epoch: 5	[48640/60000 (81%)]	Loss: 112.017105
Train Epoch: 5	[49920/60000 (83%)]	Loss: 107.457336
Train Epoch: 5	[51200/60000 (85%)]	Loss: 108.506172
Train Epoch: 5	[52480/60000 (87%)]	Loss: 110.043839
Train Epoch: 5	[53760/60000 (90%)]	Loss: 109.074814
Train Epoch: 5	[55040/60000 (92%)]	Loss: 110.417053
Train Epoch: 5	[56320/60000 (94%)]	Loss: 109.405983
Train Epoch: 5	[57600/60000 (96%)]	Loss: 110.304794
Train Epoch: 5	[58880/60000 (98%)]	Loss: 106.000778
====> Epoch: 5 Average loss: 109.7647		
====> Test set loss: 108.5804		
Train Epoch: 6	[0/60000 (0%)]	Loss: 108.792473
Train Epoch: 6	[1280/60000 (2%)]	Loss: 109.843498
Train Epoch: 6	[2560/60000 (4%)]	Loss: 106.755135
Train Epoch: 6	[3840/60000 (6%)]	Loss: 109.283646
Train Epoch: 6	[5120/60000 (9%)]	Loss: 103.981186
Train Epoch: 6	[6400/60000 (11%)]	Loss: 108.710709
Train Epoch: 6	[7680/60000 (13%)]	Loss: 108.621933
Train Epoch: 6	[8960/60000 (15%)]	Loss: 107.978188
Train Epoch: 6	[10240/60000 (17%)]	Loss: 108.636398
Train Epoch: 6	[11520/60000 (19%)]	Loss: 109.177582
Train Epoch: 6	[12800/60000 (21%)]	Loss: 109.655365
Train Epoch: 6	[14080/60000 (23%)]	Loss: 103.551468
Train Epoch: 6	[15360/60000 (26%)]	Loss: 110.711349
Train Epoch: 6	[16640/60000 (28%)]	Loss: 108.009995
Train Epoch: 6	[17920/60000 (30%)]	Loss: 105.902206
Train Epoch: 6	[19200/60000 (32%)]	Loss: 107.921539
Train Epoch: 6	[20480/60000 (34%)]	Loss: 107.570740
Train Epoch: 6	[21760/60000 (36%)]	Loss: 108.233551
Train Epoch: 6	[23040/60000 (38%)]	Loss: 106.585266
Train Epoch: 6	[24320/60000 (41%)]	Loss: 111.381287
Train Epoch: 6	[25600/60000 (43%)]	Loss: 109.907730
Train Epoch: 6	[26880/60000 (45%)]	Loss: 106.212189
Train Epoch: 6	[28160/60000 (47%)]	Loss: 110.113678
Train Epoch: 6	[29440/60000 (49%)]	Loss: 106.747978
Train Epoch: 6	[30720/60000 (51%)]	Loss: 109.891777
Train Epoch: 6	[32000/60000 (53%)]	Loss: 112.399368
Train Epoch: 6	[33280/60000 (55%)]	Loss: 109.839218
Train Epoch: 6	[34560/60000 (58%)]	Loss: 108.522247
Train Epoch: 6	[35840/60000 (60%)]	Loss: 107.023788
Train Epoch: 6	[37120/60000 (62%)]	Loss: 105.128044
Train Epoch: 6	[38400/60000 (64%)]	Loss: 108.024330
Train Epoch: 6	[39680/60000 (66%)]	Loss: 108.783821
Train Epoch: 6	[40960/60000 (68%)]	Loss: 108.735832
Train Epoch: 6	[42240/60000 (70%)]	Loss: 111.266647
Train Epoch: 6	[43520/60000 (72%)]	Loss: 108.405220

Train Epoch: 6	[44800/60000 (75%)]	Loss: 110.589531
Train Epoch: 6	[46080/60000 (77%)]	Loss: 108.927223
Train Epoch: 6	[47360/60000 (79%)]	Loss: 110.609024
Train Epoch: 6	[48640/60000 (81%)]	Loss: 111.255577
Train Epoch: 6	[49920/60000 (83%)]	Loss: 107.303322
Train Epoch: 6	[51200/60000 (85%)]	Loss: 109.771217
Train Epoch: 6	[52480/60000 (87%)]	Loss: 108.018066
Train Epoch: 6	[53760/60000 (90%)]	Loss: 106.892563
Train Epoch: 6	[55040/60000 (92%)]	Loss: 109.486618
Train Epoch: 6	[56320/60000 (94%)]	Loss: 108.837326
Train Epoch: 6	[57600/60000 (96%)]	Loss: 107.978882
Train Epoch: 6	[58880/60000 (98%)]	Loss: 106.022835
====>	Epoch: 6 Average loss: 108.6533	
====>	Test set loss: 107.3521	
Train Epoch: 7	[0/60000 (0%)]	Loss: 105.753067
Train Epoch: 7	[1280/60000 (2%)]	Loss: 109.807709
Train Epoch: 7	[2560/60000 (4%)]	Loss: 106.707474
Train Epoch: 7	[3840/60000 (6%)]	Loss: 106.157364
Train Epoch: 7	[5120/60000 (9%)]	Loss: 109.868050
Train Epoch: 7	[6400/60000 (11%)]	Loss: 103.690460
Train Epoch: 7	[7680/60000 (13%)]	Loss: 111.863525
Train Epoch: 7	[8960/60000 (15%)]	Loss: 111.809479
Train Epoch: 7	[10240/60000 (17%)]	Loss: 108.274254
Train Epoch: 7	[11520/60000 (19%)]	Loss: 105.662521
Train Epoch: 7	[12800/60000 (21%)]	Loss: 108.756439
Train Epoch: 7	[14080/60000 (23%)]	Loss: 110.899017
Train Epoch: 7	[15360/60000 (26%)]	Loss: 105.090645
Train Epoch: 7	[16640/60000 (28%)]	Loss: 113.774933
Train Epoch: 7	[17920/60000 (30%)]	Loss: 104.258972
Train Epoch: 7	[19200/60000 (32%)]	Loss: 108.107628
Train Epoch: 7	[20480/60000 (34%)]	Loss: 109.680389
Train Epoch: 7	[21760/60000 (36%)]	Loss: 109.086113
Train Epoch: 7	[23040/60000 (38%)]	Loss: 107.039391
Train Epoch: 7	[24320/60000 (41%)]	Loss: 108.696823
Train Epoch: 7	[25600/60000 (43%)]	Loss: 105.003456
Train Epoch: 7	[26880/60000 (45%)]	Loss: 107.796143
Train Epoch: 7	[28160/60000 (47%)]	Loss: 107.382065
Train Epoch: 7	[29440/60000 (49%)]	Loss: 105.061134
Train Epoch: 7	[30720/60000 (51%)]	Loss: 106.721886
Train Epoch: 7	[32000/60000 (53%)]	Loss: 106.639496
Train Epoch: 7	[33280/60000 (55%)]	Loss: 109.197266
Train Epoch: 7	[34560/60000 (58%)]	Loss: 103.344543
Train Epoch: 7	[35840/60000 (60%)]	Loss: 107.260941
Train Epoch: 7	[37120/60000 (62%)]	Loss: 108.830856
Train Epoch: 7	[38400/60000 (64%)]	Loss: 105.987488
Train Epoch: 7	[39680/60000 (66%)]	Loss: 105.952301
Train Epoch: 7	[40960/60000 (68%)]	Loss: 108.500664
Train Epoch: 7	[42240/60000 (70%)]	Loss: 104.247780
Train Epoch: 7	[43520/60000 (72%)]	Loss: 108.441513
Train Epoch: 7	[44800/60000 (75%)]	Loss: 106.121536
Train Epoch: 7	[46080/60000 (77%)]	Loss: 110.225761
Train Epoch: 7	[47360/60000 (79%)]	Loss: 107.048126
Train Epoch: 7	[48640/60000 (81%)]	Loss: 104.449242
Train Epoch: 7	[49920/60000 (83%)]	Loss: 108.693619
Train Epoch: 7	[51200/60000 (85%)]	Loss: 106.837967
Train Epoch: 7	[52480/60000 (87%)]	Loss: 106.696686

Train Epoch: 7	[53760/60000 (90%)]	Loss: 105.937592
Train Epoch: 7	[55040/60000 (92%)]	Loss: 107.854240
Train Epoch: 7	[56320/60000 (94%)]	Loss: 109.199646
Train Epoch: 7	[57600/60000 (96%)]	Loss: 106.099258
Train Epoch: 7	[58880/60000 (98%)]	Loss: 109.020370
====> Epoch: 7 Average loss: 107.7255		
====> Test set loss: 106.9262		
Train Epoch: 8	[0/60000 (0%)]	Loss: 104.886459
Train Epoch: 8	[1280/60000 (2%)]	Loss: 112.651688
Train Epoch: 8	[2560/60000 (4%)]	Loss: 108.359314
Train Epoch: 8	[3840/60000 (6%)]	Loss: 105.984772
Train Epoch: 8	[5120/60000 (9%)]	Loss: 107.790588
Train Epoch: 8	[6400/60000 (11%)]	Loss: 107.119354
Train Epoch: 8	[7680/60000 (13%)]	Loss: 107.585602
Train Epoch: 8	[8960/60000 (15%)]	Loss: 109.636856
Train Epoch: 8	[10240/60000 (17%)]	Loss: 108.422661
Train Epoch: 8	[11520/60000 (19%)]	Loss: 105.837891
Train Epoch: 8	[12800/60000 (21%)]	Loss: 111.285767
Train Epoch: 8	[14080/60000 (23%)]	Loss: 106.126617
Train Epoch: 8	[15360/60000 (26%)]	Loss: 108.006294
Train Epoch: 8	[16640/60000 (28%)]	Loss: 104.489143
Train Epoch: 8	[17920/60000 (30%)]	Loss: 109.457153
Train Epoch: 8	[19200/60000 (32%)]	Loss: 110.136887
Train Epoch: 8	[20480/60000 (34%)]	Loss: 108.636307
Train Epoch: 8	[21760/60000 (36%)]	Loss: 103.688026
Train Epoch: 8	[23040/60000 (38%)]	Loss: 109.066711
Train Epoch: 8	[24320/60000 (41%)]	Loss: 105.818947
Train Epoch: 8	[25600/60000 (43%)]	Loss: 105.486839
Train Epoch: 8	[26880/60000 (45%)]	Loss: 104.062836
Train Epoch: 8	[28160/60000 (47%)]	Loss: 109.817207
Train Epoch: 8	[29440/60000 (49%)]	Loss: 106.121277
Train Epoch: 8	[30720/60000 (51%)]	Loss: 109.540382
Train Epoch: 8	[32000/60000 (53%)]	Loss: 110.190193
Train Epoch: 8	[33280/60000 (55%)]	Loss: 105.971313
Train Epoch: 8	[34560/60000 (58%)]	Loss: 111.945938
Train Epoch: 8	[35840/60000 (60%)]	Loss: 106.881721
Train Epoch: 8	[37120/60000 (62%)]	Loss: 104.448540
Train Epoch: 8	[38400/60000 (64%)]	Loss: 107.034019
Train Epoch: 8	[39680/60000 (66%)]	Loss: 105.195847
Train Epoch: 8	[40960/60000 (68%)]	Loss: 107.630096
Train Epoch: 8	[42240/60000 (70%)]	Loss: 103.705788
Train Epoch: 8	[43520/60000 (72%)]	Loss: 107.277260
Train Epoch: 8	[44800/60000 (75%)]	Loss: 106.913818
Train Epoch: 8	[46080/60000 (77%)]	Loss: 103.027283
Train Epoch: 8	[47360/60000 (79%)]	Loss: 105.682724
Train Epoch: 8	[48640/60000 (81%)]	Loss: 105.820496
Train Epoch: 8	[49920/60000 (83%)]	Loss: 104.515793
Train Epoch: 8	[51200/60000 (85%)]	Loss: 106.119202
Train Epoch: 8	[52480/60000 (87%)]	Loss: 106.833763
Train Epoch: 8	[53760/60000 (90%)]	Loss: 109.278282
Train Epoch: 8	[55040/60000 (92%)]	Loss: 104.832031
Train Epoch: 8	[56320/60000 (94%)]	Loss: 108.416016
Train Epoch: 8	[57600/60000 (96%)]	Loss: 107.734261
Train Epoch: 8	[58880/60000 (98%)]	Loss: 105.834267
====> Epoch: 8 Average loss: 107.1412		
====> Test set loss: 106.4784		


```

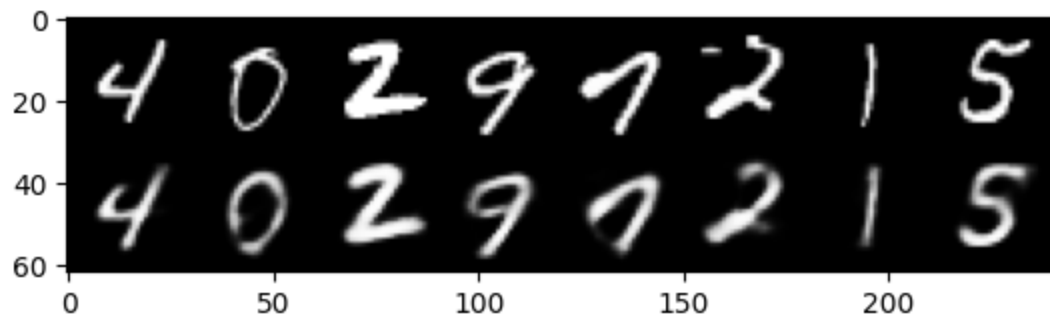
Train Epoch: 9 [0/60000 (0%)] Loss: 108.956207
Train Epoch: 9 [1280/60000 (2%)] Loss: 110.003235
Train Epoch: 9 [2560/60000 (4%)] Loss: 105.619370
Train Epoch: 9 [3840/60000 (6%)] Loss: 111.411957
Train Epoch: 9 [5120/60000 (9%)] Loss: 108.191467
Train Epoch: 9 [6400/60000 (11%)] Loss: 105.870956
Train Epoch: 9 [7680/60000 (13%)] Loss: 109.403412
Train Epoch: 9 [8960/60000 (15%)] Loss: 109.041954
Train Epoch: 9 [10240/60000 (17%)] Loss: 108.411545
Train Epoch: 9 [11520/60000 (19%)] Loss: 109.678894
Train Epoch: 9 [12800/60000 (21%)] Loss: 105.038376
Train Epoch: 9 [14080/60000 (23%)] Loss: 107.124405
Train Epoch: 9 [15360/60000 (26%)] Loss: 109.680435
Train Epoch: 9 [16640/60000 (28%)] Loss: 106.943916
Train Epoch: 9 [17920/60000 (30%)] Loss: 107.828590
Train Epoch: 9 [19200/60000 (32%)] Loss: 102.476265
Train Epoch: 9 [20480/60000 (34%)] Loss: 105.036598
Train Epoch: 9 [21760/60000 (36%)] Loss: 112.019249
Train Epoch: 9 [23040/60000 (38%)] Loss: 106.408478
Train Epoch: 9 [24320/60000 (41%)] Loss: 101.955185
Train Epoch: 9 [25600/60000 (43%)] Loss: 109.354767
Train Epoch: 9 [26880/60000 (45%)] Loss: 107.907532
Train Epoch: 9 [28160/60000 (47%)] Loss: 106.974899
Train Epoch: 9 [29440/60000 (49%)] Loss: 104.463737
Train Epoch: 9 [30720/60000 (51%)] Loss: 109.173538
Train Epoch: 9 [32000/60000 (53%)] Loss: 111.678703
Train Epoch: 9 [33280/60000 (55%)] Loss: 108.161697
Train Epoch: 9 [34560/60000 (58%)] Loss: 106.469955
Train Epoch: 9 [35840/60000 (60%)] Loss: 106.338165
Train Epoch: 9 [37120/60000 (62%)] Loss: 105.757660
Train Epoch: 9 [38400/60000 (64%)] Loss: 103.441078
Train Epoch: 9 [39680/60000 (66%)] Loss: 105.653290
Train Epoch: 9 [40960/60000 (68%)] Loss: 109.529114
Train Epoch: 9 [42240/60000 (70%)] Loss: 106.665344
Train Epoch: 9 [43520/60000 (72%)] Loss: 102.673630
Train Epoch: 9 [44800/60000 (75%)] Loss: 103.667542
Train Epoch: 9 [46080/60000 (77%)] Loss: 110.559036
Train Epoch: 9 [47360/60000 (79%)] Loss: 106.611153
Train Epoch: 9 [48640/60000 (81%)] Loss: 108.339844
Train Epoch: 9 [49920/60000 (83%)] Loss: 107.841591
Train Epoch: 9 [51200/60000 (85%)] Loss: 106.361755
Train Epoch: 9 [52480/60000 (87%)] Loss: 103.844276
Train Epoch: 9 [53760/60000 (90%)] Loss: 100.920700
Train Epoch: 9 [55040/60000 (92%)] Loss: 102.263672
Train Epoch: 9 [56320/60000 (94%)] Loss: 104.389992
Train Epoch: 9 [57600/60000 (96%)] Loss: 106.251175
Train Epoch: 9 [58880/60000 (98%)] Loss: 101.995880
====> Epoch: 9 Average loss: 106.6536
====> Test set loss: 105.7770
Train Epoch: 10 [0/60000 (0%)] Loss: 105.616417
Train Epoch: 10 [1280/60000 (2%)] Loss: 104.073997
Train Epoch: 10 [2560/60000 (4%)] Loss: 110.331886
Train Epoch: 10 [3840/60000 (6%)] Loss: 109.129852
Train Epoch: 10 [5120/60000 (9%)] Loss: 107.652718
Train Epoch: 10 [6400/60000 (11%)] Loss: 111.047997
Train Epoch: 10 [7680/60000 (13%)] Loss: 106.972534

```

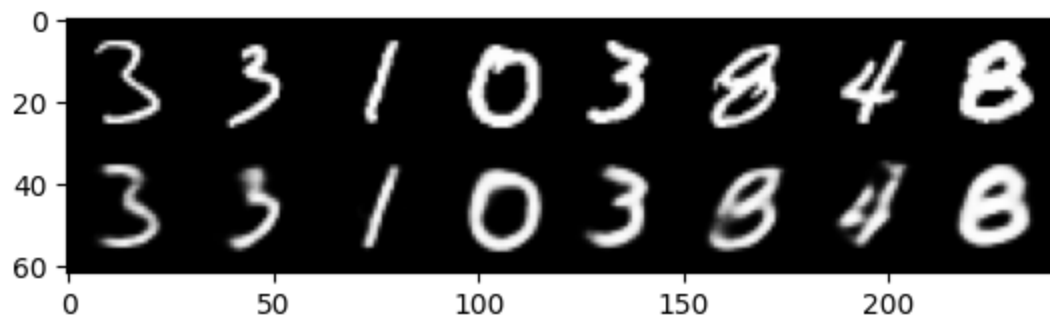
Train Epoch: 10	[8960/60000 (15%)]	Loss: 104.020187
Train Epoch: 10	[10240/60000 (17%)]	Loss: 107.539360
Train Epoch: 10	[11520/60000 (19%)]	Loss: 104.985855
Train Epoch: 10	[12800/60000 (21%)]	Loss: 106.095139
Train Epoch: 10	[14080/60000 (23%)]	Loss: 103.864082
Train Epoch: 10	[15360/60000 (26%)]	Loss: 110.078468
Train Epoch: 10	[16640/60000 (28%)]	Loss: 106.947792
Train Epoch: 10	[17920/60000 (30%)]	Loss: 104.476013
Train Epoch: 10	[19200/60000 (32%)]	Loss: 105.516968
Train Epoch: 10	[20480/60000 (34%)]	Loss: 105.262489
Train Epoch: 10	[21760/60000 (36%)]	Loss: 104.070953
Train Epoch: 10	[23040/60000 (38%)]	Loss: 108.599854
Train Epoch: 10	[24320/60000 (41%)]	Loss: 104.694016
Train Epoch: 10	[25600/60000 (43%)]	Loss: 102.192734
Train Epoch: 10	[26880/60000 (45%)]	Loss: 105.900627
Train Epoch: 10	[28160/60000 (47%)]	Loss: 107.203812
Train Epoch: 10	[29440/60000 (49%)]	Loss: 105.808884
Train Epoch: 10	[30720/60000 (51%)]	Loss: 107.240921
Train Epoch: 10	[32000/60000 (53%)]	Loss: 109.982513
Train Epoch: 10	[33280/60000 (55%)]	Loss: 103.284653
Train Epoch: 10	[34560/60000 (58%)]	Loss: 107.142319
Train Epoch: 10	[35840/60000 (60%)]	Loss: 107.955704
Train Epoch: 10	[37120/60000 (62%)]	Loss: 106.441566
Train Epoch: 10	[38400/60000 (64%)]	Loss: 109.615349
Train Epoch: 10	[39680/60000 (66%)]	Loss: 104.431282
Train Epoch: 10	[40960/60000 (68%)]	Loss: 105.173721
Train Epoch: 10	[42240/60000 (70%)]	Loss: 104.376266
Train Epoch: 10	[43520/60000 (72%)]	Loss: 103.606094
Train Epoch: 10	[44800/60000 (75%)]	Loss: 108.285400
Train Epoch: 10	[46080/60000 (77%)]	Loss: 105.771667
Train Epoch: 10	[47360/60000 (79%)]	Loss: 103.607437
Train Epoch: 10	[48640/60000 (81%)]	Loss: 107.357559
Train Epoch: 10	[49920/60000 (83%)]	Loss: 106.942848
Train Epoch: 10	[51200/60000 (85%)]	Loss: 101.722122
Train Epoch: 10	[52480/60000 (87%)]	Loss: 105.944931
Train Epoch: 10	[53760/60000 (90%)]	Loss: 106.094002
Train Epoch: 10	[55040/60000 (92%)]	Loss: 102.798187
Train Epoch: 10	[56320/60000 (94%)]	Loss: 103.454109
Train Epoch: 10	[57600/60000 (96%)]	Loss: 107.065506
Train Epoch: 10	[58880/60000 (98%)]	Loss: 105.683838
====> Epoch: 10 Average loss: 106.1477		
====> Test set loss: 105.4860		

Run the following cells to display the results for grading:

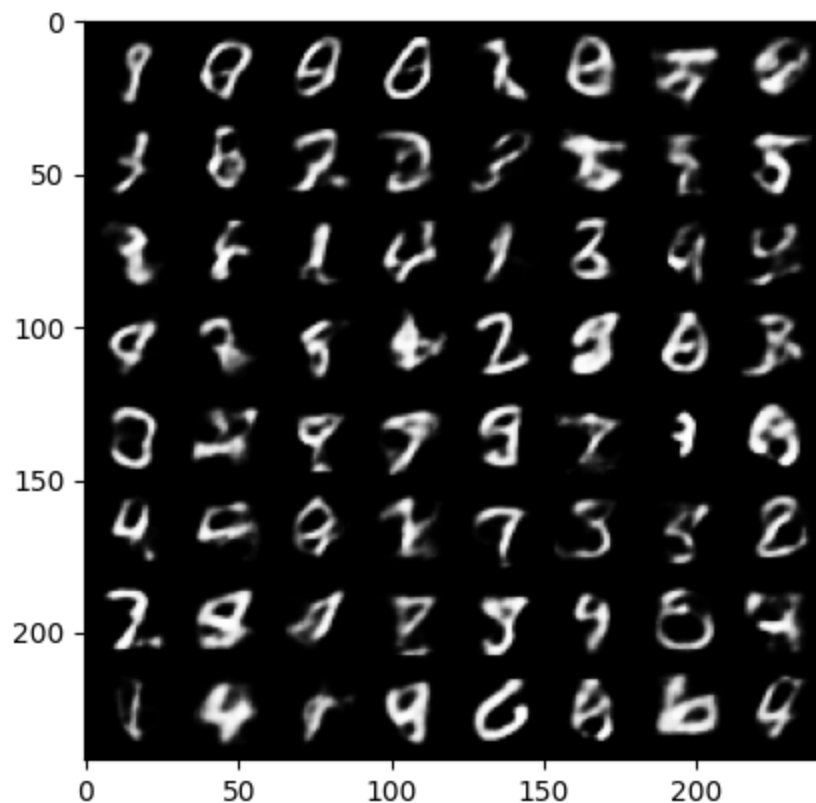
```
In [15]: img = mpimg.imread('results/reconstruction_5.png')
plt.imshow(img)
plt.show()
```



```
In [16]: img = mpimg.imread('results/reconstruction_10.png')
plt.imshow(img)
plt.show()
```



```
In [17]: img = mpimg.imread('results/sample_10.png')
plt.imshow(img)
plt.show()
```



Run the following to download the files from Google Colab

```
In [18]: from google.colab import files
!zip -r /content/results.zip /content/results
files.download("/content/results.zip")

adding: content/results/ (stored 0%)
adding: content/results/reconstruction_1.png (deflated 3%)
adding: content/results/sample_2.png (deflated 3%)
adding: content/results/sample_1.png (deflated 2%)
adding: content/results/sample_3.png (deflated 3%)
adding: content/results/sample_10.png (deflated 4%)
adding: content/results/reconstruction_7.png (deflated 3%)
adding: content/results/sample_9.png (deflated 4%)
adding: content/results/reconstruction_5.png (deflated 3%)
adding: content/results/reconstruction_3.png (deflated 3%)
adding: content/results/reconstruction_9.png (deflated 3%)
adding: content/results/sample_6.png (deflated 4%)
adding: content/results/sample_8.png (deflated 3%)
adding: content/results/sample_7.png (deflated 4%)
adding: content/results/sample_4.png (deflated 3%)
adding: content/results/reconstruction_8.png (deflated 3%)
adding: content/results/reconstruction_10.png (deflated 3%)
adding: content/results/reconstruction_4.png (deflated 3%)
adding: content/results/reconstruction_2.png (deflated 3%)
adding: content/results/reconstruction_6.png (deflated 4%)
adding: content/results/sample_5.png (deflated 3%)
```

Problem 3: Self-Attention for Sequence Prediction

From Sequential to Parallel

In Problem 1, we used an LSTM to process sequences *step by step*, maintaining a hidden state that serves as memory. While effective, this sequential processing has a fundamental limitation: information from early timesteps must pass through many intermediate steps to influence later predictions, which can lead to information loss (the vanishing gradient problem).

Attention offers an alternative paradigm: instead of processing sequentially, we *look at all timesteps at once* and learn which ones are most important for making each prediction. This is the core idea behind the "[Attention Is All You Need](#)" paper (Vaswani et al., 2017), which introduced the Transformer architecture that powers modern language models (GPT, BERT, etc.). Since then, transformers are a staple in the fields of robotics, NLP, computer vision, sound processing etc. It doesn't lose "memory" like LSTMs, but scales so elegantly, and has formed the foundation of "scaling laws".

Scaled Dot-Product Attention

The attention mechanism uses three components: **Queries** (Q), **Keys** (K), and **Values** (V). An intuitive analogy is a library search:

- **Query**: what you are looking for (your search terms)
- **Key**: the label/index on each book (what each item offers)
- **Value**: the actual content of each book (the information you retrieve)

The attention mechanism computes a weighted sum of Values, where the weight for each Value is determined by how well the corresponding Key matches the Query:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

where d_k is the dimension of the key vectors. The scaling factor $\frac{1}{\sqrt{d_k}}$ is critical: without it, the dot products QK^T grow large in magnitude as d_k increases, pushing the softmax function into regions where it has extremely small gradients, effectively preventing learning.



Multi-Head Attention

Rather than performing a single attention function, *multi-head attention* runs the attention mechanism multiple times in parallel with different learned linear projections of Q , K , and V . Each "head" can learn to attend to different types of relationships in the data (e.g., one head might focus on nearby timesteps while another captures long-range periodicity). The outputs of all heads are concatenated and linearly projected to produce the final result.

References

- [Attention Is All You Need \(Vaswani et al., 2017\)](#)
- [The Illustrated Transformer \(Jay Alammar\)](#)
- [PyTorch MultiheadAttention documentation](#)

Your Problem: Attention-Based Sequence Prediction

In this problem, you will build a **self-attention model** to predict the **same sine wave sequences from Problem 1**. This allows a direct comparison between the LSTM approach (sequential memory) and the attention approach (parallel, weighted lookups, arguably simpler!).

Your model architecture:

$$x \rightarrow \text{Linear}(1, d_{\text{model}}) \rightarrow \text{Self-Attention}(Q, K, V) \rightarrow \text{Linear}(d_{\text{model}}, 1) \rightarrow y$$

You will implement:

1. The Q, K, V projection layers
2. The `scaled_dot_product_attention` function (compute scores, apply softmax, weighted sum)
3. The forward pass wiring these components together

After training, you will visualize the **attention weight heatmap** to see which input timesteps the model "attends to" when making each prediction.

Your Code:

```
In [1]: def student_function_3(data, lr=0.001, training_steps=75, seq_len=50, pred_len=10):
        """
        Description: This function implements a self-attention model for sequence prediction.
        Uses the same sine wave data as Problem 1, but instead of sequential LSTM,
        the model uses scaled dot-product attention to look at all timesteps at once.

        Parameters
        -----
        data : np.ndarray
            input data (sine wave sequences), shape (N, L)
        lr : float
            learning rate for Adam optimizer
        training_steps : int
            number of training epochs -- you can reduce this number for faster training
        seq_len : int
            number of input timesteps to use as context
        pred_len : int
            number of future timesteps to predict
        d_model : int
            dimension of the model's internal representations

        Returns
        -----
        predictions : np.ndarray
            predicted sequences
        attn_weights : np.ndarray
            attention weights for visualization (shape: seq_len x seq_len)
        """
        #raise NotImplementedError # comment this out once implemented

        def scaled_dot_product_attention(Q, K, V):
```

```
"""
```

```
Compute scaled dot-product attention.
```

```
Recall the formula from the problem description:
```

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$$

```
Args:
```

```
Q: Queries, shape (batch, seq_len, d_k)
```

```
K: Keys, shape (batch, seq_len, d_k)
```

```
V: Values, shape (batch, seq_len, d_k)
```

```
Returns:
```

```
output: weighted sum of values, shape (batch, seq_len, d_k)
```

```
attn_weights: attention weights, shape (batch, seq_len, seq_len)
```

```
"""
```

```
d_k = Q.size(-1)
```

```
# Step 1: Compute raw attention scores.
```

```
# You need to multiply Q by K^T and scale by sqrt(d_k).
```

```
# Remember these are batched tensors – use the appropriate
```

```
# batched matrix multiplication from PyTorch.
```

```
scores = torch.bmm(Q, K.transpose(1, 2)) / (d_k ** 0.5) # Fill this
```

```
# Step 2: Convert scores to a probability distribution (attention we
```

```
# Which dimension should you normalize over, and why?
```

```
attn_weights = F.softmax(scores, dim=-1) # Fill this in
```

```
# Step 3: Use the attention weights to compute a weighted combinatio
```

```
output = torch.bmm(attn_weights, V) # Fill this in
```

```
return output, attn_weights
```

```
class AttentionSeqModel(nn.Module):
```

```
    def __init__(self, d_model=64):
```

```
        super(AttentionSeqModel, self).__init__()
```

```
        self.d_model = d_model
```

```
        self.pos_emb = nn.Parameter(torch.zeros(1, seq_len, d_model))
```

```
# Project scalar input to d_model dimensions (provided)
```

```
self.input_proj = nn.Linear(1, d_model)
```

```
# Define the Q, K, V projection layers.
```

```
# Each should map from d_model dimensions to d_model dimensions.
```

```
self.W_Q = nn.Linear(d_model, d_model) # Fill this in
```

```
self.W_K = nn.Linear(d_model, d_model) # Fill this in
```

```
self.W_V = nn.Linear(d_model, d_model) # Fill this in
```

```
# Output projection (provided): maps from d_model back to scalar
```

```
self.output_proj = nn.Linear(d_model, 1)
```

```
    def forward(self, x):
```

```
        """
```

```
        Forward pass of the attention model.
```

```
        Args:
```

```
            x: input sequence, shape (batch, seq_len, 1)
```

Returns:

```
    output: predicted values, shape (batch, seq_len, 1)
    attn_weights: attention weight matrix, shape (batch, seq_len, seq_len)

# Project input to d_model dimensions (provided)
#x_proj = self.input_proj(x) # (batch, seq_len, d_model)
x_proj = self.input_proj(x) + self.pos_emb

# Produce Q, K, V from the projected input using the layers above
# In self-attention, Q, K, and V all come from the same source
# but are transformed through different learned projections.
Q = self.W_Q(x_proj) # Fill this in
K = self.W_K(x_proj) # Fill this in
V = self.W_V(x_proj) # Fill this in

#added for troubleshooting
S = x.shape[1]
causal_mask = torch.triu(
    torch.ones(S, S, device=x.device, dtype=torch.bool),
    diagonal=1
).unsqueeze(0) # (1, S, S)

# Apply scaled dot-product attention (provided call)
attn_output, attn_weights = scaled_dot_product_attention(Q, K, V, causal_mask)

# Project back to scalar output (provided)
output = self.output_proj(attn_output) # (batch, seq_len, 1)

return output, attn_weights

# ---- Data Preparation (provided) ----
# Create input-target pairs from the sine wave data
# Input: seq_len timesteps, Target: the next pred_len timesteps shifted
N, L = data.shape
inputs = []
targets = []
for i in range(N):
    for j in range(0, L - seq_len - pred_len, pred_len):
        inputs.append(data[i, j:j+seq_len])
        targets.append(data[i, j+1:j+seq_len+1]) # shifted by 1

inputs = torch.FloatTensor(np.array(inputs)).unsqueeze(-1) # (num_samples, seq_len, 1)
targets = torch.FloatTensor(np.array(targets)).unsqueeze(-1) # (num_samples, pred_len, 1)

# ---- Training (provided) ----
model = AttentionSeqModel(d_model=d_model)
optimizer = optim.Adam(model.parameters(), lr=lr)
criterion = nn.MSELoss()

model.train()
for epoch in range(training_steps):
    optimizer.zero_grad()
    pred, attn_w = model(inputs)
    loss = criterion(pred, targets)
    loss.backward()
    optimizer.step()
```



```

        if (epoch + 1) % 50 == 0:
            print(f'Epoch {epoch+1}/{training_steps}, Loss: {loss.item():.6f}')

    # ---- Extract predictions and attention weights (provided) ----
    model.eval()
    with torch.no_grad():
        # Use the first 3 sequences for visualization (like Problem 1)
        test_inputs = []
        for i in range(3):
            test_inputs.append(data[i, :seq_len])
        test_tensor = torch.FloatTensor(np.array(test_inputs)).unsqueeze(-1)
        predictions, attn_weights = model(test_tensor)

    return predictions.squeeze(-1).numpy(), attn_weights[0].numpy()

```

Evaluation

```

In [6]: # Generate sine wave data (same as Problem 1)
np.random.seed(2)
T = 20
L = 1000
N = 100

x = np.empty((N, L), 'int64')
x[:, :] = np.array(range(L)) + np.random.randint(-4, 4, N).reshape(N, 1)
data = np.sin(x / 1.0 / T).astype('float64')

# Call your attention model
predictions_3, attn_weights_3 = student_function_3(data)

```

Epoch 50/75, Loss: 0.053572

Visualize Attention Weights

```

In [7]: # Plot 1: Predictions vs Ground Truth
fig, axes = plt.subplots(1, 2, figsize=(18, 6))

# Left plot: predictions vs ground truth
seq_len = predictions_3.shape[1]
colors = ['r', 'g', 'b']
for i in range(3):
    ground_truth = data[i, :seq_len]
    axes[0].plot(ground_truth, colors[i] + '--', linewidth=1.5, label=f'Ground Truth')
    axes[0].plot(predictions_3[i], colors[i], linewidth=1.5, label=f'Predictions')
axes[0].set_title('Attention Model: Predictions vs Ground Truth', fontsize=14)
axes[0].set_xlabel('Timestep', fontsize=12)
axes[0].set_ylabel('Value', fontsize=12)
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Right plot: Attention weight heatmap
im = axes[1].imshow(attn_weights_3, cmap='viridis', aspect='auto')
axes[1].set_title('Attention Weights Heatmap', fontsize=14)
axes[1].set_xlabel('Key Position (input timestep)', fontsize=12)

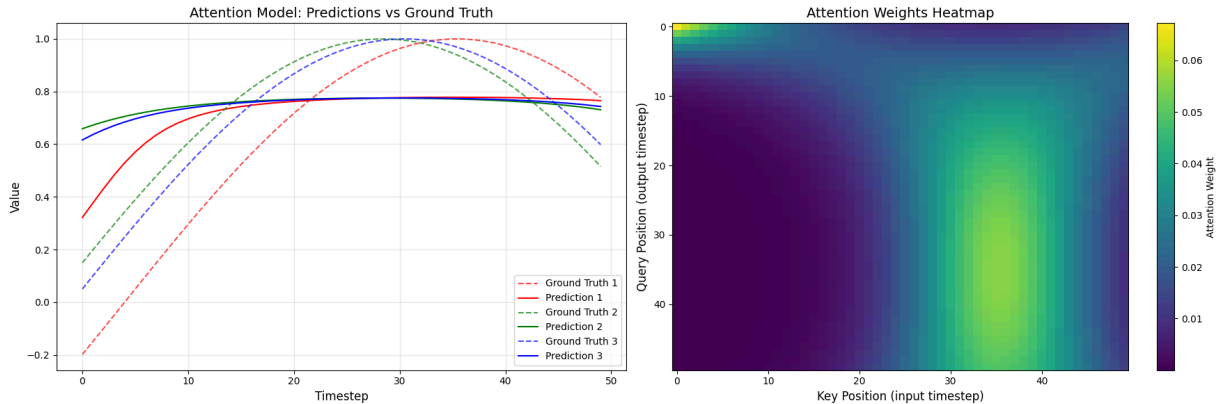
```

```

axes[1].set_ylabel('Query Position (output timestep)', fontsize=12)
plt.colorbar(im, ax=axes[1], label='Attention Weight')

plt.tight_layout()
plt.savefig('attention_results.pdf')
plt.show()
print("Attention weights shape:", attn_weights_3.shape)
print("Notice how the attention pattern shows which input timesteps influence

```



Attention weights shape: (50, 50)

Notice how the attention pattern shows which input timesteps influence each output prediction.

Bonus Question: Attention Beyond Text Sequences

In this problem, we applied self-attention to **time-series data** (sine wave sequences). BUT, the attention mechanism is remarkably versatile and has been adopted across many different data modalities beyond text and simple 1D sequences.

Your task: Name **one concrete example** of how attention (or Transformers) has been applied to a **different data modality** relevant to robotics or mechanical engineering or computer science. Briefly describe (2-3 sentences):

1. What is the data modality? (e.g., force/torque signals, point clouds, multi-agent trajectories, etc.)

Answer = One data modality is 3D point clouds from a robot's depth sensor (RGB-D camera or LiDAR), used for perception tasks like object segmentation, pose estimation, and scene understanding.

2. What role does attention play in that application? (e.g., what is it "attending to" and why is that useful?)

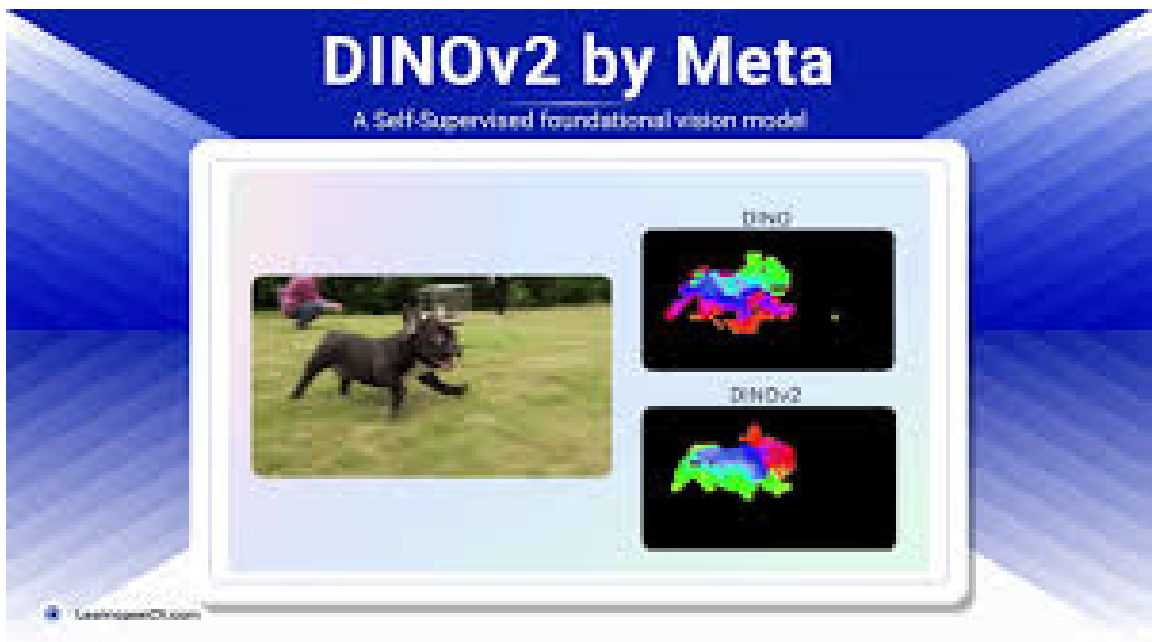
Answer = Attention lets each point attend to other points (often nearby in 3D space) to learn which parts of the cloud are most informative. This helps the model capture both local geometry (surfaces/edges) and long-range structure, so the robot can focus on relevant regions for grasping or navigation instead of treating all points equally.

*Hint: Think about the types of sequential or structured data a robot encounters — contact force profiles during manipulation, 3D point clouds from depth sensors

Problem 4: DINOv2 Vision Feature Exploration

Vision Transformers (ViTs)

Vision Transformers apply the Transformer architecture (from Problem 3) to images. The key idea is to split an image into fixed-size **patches** — analogous to words in a sentence — and process them with self-attention. Each patch is linearly embedded into a vector, and these patch embeddings are processed by a standard Transformer encoder. For example, the tokens of the top of an image (clouds, sky), can affect the tokens at the bottom of an image (beach, ground, not higher sky) thanks to attention.



For example, with a **patch size of 14 pixels** and an input image of 518×518 pixels, we obtain a $37 \times 37 = 1369$ grid of patch tokens, plus one special **[CLS]** token that summarizes the entire image.

DINOv2: Self-Supervised Visual Features

DINOv2 (Oquab et al., 2023) is a self-supervised pretrained Vision Transformer from Meta AI Research. It is trained **without any labels** using a self-distillation objective: a student network learns to match the output of a teacher network (an exponential moving average of the student) on different augmented views of the same image. For example, a photo of a bear closer up is the same as a photo of the bear farther away, right? The features should be similar.

The result is a model that produces **rich, semantically meaningful patch-level features** — patches depicting similar semantic content (e.g., two parts of a dog's fur, or

two regions of sky) will have similar feature vectors, even without any supervised training. No labels -- it just works well!! And since there are smaller versions of this model, one might call it a "free gift".

Cosine Similarity

Each patch token from DINOv2 is a feature vector (384 dimensions for ViT-S/14). We measure similarity using **cosine similarity**:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|}$$

which ranges from -1 (opposite) to 1 (identical direction). Patches with high cosine similarity represent semantically similar regions.

References

- [DINOv2: Learning Robust Visual Features without Supervision](#)
- [An Image is Worth 16x16 Words \(ViT paper\)](#)
- [PyTorch Hub: DINOv2](#)

Your Problem: Interactive Feature Visualization

In this problem, you will:

1. Load a pretrained DINOv2 model (`dinov2_vits14`) using `torch.hub`
2. Extract patch token features from a sample image
3. Compute cosine similarity between a reference patch and all other patches
4. Build an interactive matplotlib visualization where mousing over the image highlights semantically similar regions
5. Generate static similarity visualizations for several reference points (for PDF submission)

Your Code:

Evaluation

```
In [8]: def student_function_4(image_url):  
        """  
        Description: Loads a pretrained DINOv2 ViT-S/14 model, extracts patch tokens from an image, and provides interactive + static cosine similarity visualizations.  
  
        Parameters  
        -----  
        image_url : str  
            URL of the image to analyze
```

Returns

```
explorer : DINOv2FeatureExplorer
    """The explorer object with extracted features, for use in visualization"""
#raise NotImplementedError # comment this out once implemented

class DINOv2FeatureExplorer:
    def __init__(self):
        # Load pretrained DINOv2 ViT-S/14 model (provided)
        self.model = torch.hub.load('facebookresearch/dinov2', 'dinov2_vit_s14')
        self.model.eval()
        self.patch_size = 14
        self.feats_dim = 384 # ViT-S/14 feature dimension

        # Image preprocessing transform (provided)
        self.transform = transforms.Compose([
            transforms.Resize(518),
            transforms.CenterCrop(518),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.229, 0.229])
        ])

        # These will be set after extract_features is called
        self.patch_tokens = None
        self.grid_h = None
        self.grid_w = None
        self.original_image = None

    def extract_features(self, image):
        """
        Extract patch token features from an image using DINOv2.

        Args:
            image: PIL Image

        Sets:
            self.patch_tokens: tensor of shape (num_patches, feat_dim)
            self.grid_h, self.grid_w: spatial dimensions of the patch grid
            self.original_image: the preprocessed image for display

        Useful references:
            - DINOv2 returns a dictionary from its forward pass. Explore it to find the patch-level features (not the CLS token).
            - The model expects a 4D input tensor: (batch, channels, height, width)
            - After cropping to 518x518 with patch_size=14, think about how many patches fit along each spatial dimension.
        """
        self.original_image = image.copy()

        # Step 1: Preprocess the image into a tensor suitable for the model
        # The self.transform pipeline handles resize, crop, and normalization
        # but the model expects a batch dimension.
        img_tensor = self.transform(image).unsqueeze(0) # Fill this in
```

```

# Step 2: Pass the image through DINOv2 and extract patch token
# DINOv2's forward pass returns a dictionary with several keys.
# You need to find the correct key that contains the normalized
# (excluding the CLS token), and extract them as a 2D tensor of
# (num_patches, feat_dim).
# HINT: try printing features_dict.keys() to see what's available
with torch.no_grad():
    features_dict = self.model.forward_features(img_tensor) # F

print(features_dict.keys())

self.patch_tokens = features_dict['x_norm_patchtokens'].squeeze()

# Step 3: Compute the spatial grid dimensions.
# Given the cropped image size (518x518) and self.patch_size, calculate
# how many patches fit along the height and width.
self.grid_h = 518 // self.patch_size # Fill this in
self.grid_w = 518 // self.patch_size # Fill this in

def compute_similarity(self, patch_idx):
    """
    Compute cosine similarity between the patch at patch_idx and all
    other patches.

    Args:
        patch_idx: integer index into the flattened patch token array

    Returns:
        similarity_map: numpy array of shape (grid_h, grid_w) with cosine
        similarities
    """
    # Get the reference patch token
    ref_token = self.patch_tokens[patch_idx] # (feat_dim,)

    # Compute cosine similarity between ref_token and ALL patch tokens
    # HINT: use F.cosine_similarity(ref_token.unsqueeze(0), self.patch_tokens)
    similarities = F.cosine_similarity(ref_token.unsqueeze(0), self.patch_tokens)

    # Reshape to spatial grid (provided)
    similarity_map = similarities.numpy().reshape(self.grid_h, self.grid_w)
    return similarity_map

def pixel_to_patch(self, x, y):
    """
    Convert pixel coordinates to patch grid index.

    Args:
        x: pixel x-coordinate
        y: pixel y-coordinate

    Returns:
        patch_idx: linear index into the patch token array
    """
    # Convert pixel coords to patch grid coords
    patch_x = int(x // self.patch_size) # Fill this in
    patch_y = int(y // self.patch_size) # Fill this in

    # Clamp to valid range (provided)

```

```

        patch_x = min(patch_x, self.grid_w - 1)
        patch_y = min(patch_y, self.grid_h - 1)

        # Convert 2D grid position to linear index (provided)
        patch_idx = patch_y * self.grid_w + patch_x
        return patch_idx

    # ---- Setup (provided) ----
    # Download and load the image
    headers = {'User-Agent': 'ME326-HW4/1.0 (Stanford University; educational)'}
    response = requests.get(image_url, headers=headers)
    response.raise_for_status()
    image = Image.open(BytesIO(response.content)).convert('RGB')

    # Create explorer and extract features
    explorer = DINOv2FeatureExplorer()
    explorer.extract_features(image)

    return explorer

```

```

In [9]: # Sample image URL, of an interesting dinner table scene (no people)
image_url = "https://raw.githubusercontent.com/armlabstanford/collaborative-

# Call your function
explorer = student_function_4(image_url)
print(f"Patch tokens shape: {explorer.patch_tokens.shape}")
print(f"Grid size: {explorer.grid_h} x {explorer.grid_w}")
print(f"Total patches: {explorer.grid_h * explorer.grid_w}")

```

Downloading: "https://github.com/facebookresearch/dinov2/zipball/main" to /root/.cache/torch/hub/main.zip

```

/root/.cache/torch/hub/facebookresearch_dinov2_main/dinov2/layers/swiglu_ffn.py:51: UserWarning: xFormers is not available (SwiGLU)
  warnings.warn("xFormers is not available (SwiGLU)")
/root/.cache/torch/hub/facebookresearch_dinov2_main/dinov2/layers/attention.py:33: UserWarning: xFormers is not available (Attention)
  warnings.warn("xFormers is not available (Attention)")
/root/.cache/torch/hub/facebookresearch_dinov2_main/dinov2/layers/block.py:40: UserWarning: xFormers is not available (Block)
  warnings.warn("xFormers is not available (Block)")

```

Downloading: "https://dl.fbaipublicfiles.com/dinov2/dinov2_vits14/dinov2_vits14_pretrain.pth" to /root/.cache/torch/hub/checkpoints/dinov2_vits14_pretrain.pth

100%|██████████| 84.2M/84.2M [00:00<00:00, 300MB/s]

```

dict_keys(['x_norm_cls_token', 'x_norm_reg_tokens', 'x_norm_patch_tokens', 'x_pos_embed', 'x_norm_pos_embed', 'masks'])
Patch tokens shape: torch.Size([1369, 384])
Grid size: 37 x 37
Total patches: 1369

```

Interactive Similarity Visualization

Run the cell below to launch an interactive visualization. **Mouse over the image** on the left to see a cosine similarity heatmap on the right, showing which other patches are

most similar to the one under your cursor.

(Note: interactive mode requires a matplotlib backend like `%matplotlib notebook` or `%matplotlib qt`. If running in Colab, use the static visualization below instead.)

```
In [10]: !pip install ipympl -q

def launch_interactive(explorer):
    """Launch interactive mouse-over similarity visualization."""
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

    # Display original image on the left
    display_img = explorer.original_image.resize((518, 518))
    ax1.imshow(display_img)
    ax1.set_title('Original Image (hover over me!)', fontsize=14)
    ax1.axis('off')

    # Initialize similarity heatmap on the right
    # Start with center patch similarity
    center_idx = (explorer.grid_h // 2) * explorer.grid_w + (explorer.grid_w // 2)
    init_sim = explorer.compute_similarity(center_idx)
    heatmap = ax2.imshow(init_sim, cmap='hot', vmin=-1, vmax=1)
    ax2.set_title('Cosine Similarity Map', fontsize=14)
    ax2.axis('off')
    plt.colorbar(heatmap, ax=ax2, fraction=0.046, pad=0.04)

    # Reference point marker on the left image
    marker, = ax1.plot([], [], 'c+', markersize=20, markeredgewidth=3)

    def on_mouse_move(event):
        if event.inaxes == ax1 and event.xdata is not None and event.ydata is not None:
            x, y = int(event.xdata), int(event.ydata)
            if 0 <= x < 518 and 0 <= y < 518:
                patch_idx = explorer.pixel_to_patch(x, y)
                sim_map = explorer.compute_similarity(patch_idx)
                heatmap.set_data(sim_map)
                marker.set_data([x], [y])
                fig.canvas.draw_idle()

    fig.canvas.mpl_connect('motion_notify_event', on_mouse_move)
    plt.tight_layout()
    plt.show()

# Uncomment to run interactive mode:
# launch_interactive(explorer)
```

```
0:00 0.0/519.0 kB ? eta -:--:--
      519.0/519.0 kB 40.5 MB/s eta 0:00
0:00 0.0/1.6 MB ? eta -:--:--
      1.6/1.6 MB 90.8 MB/s eta 0:00:00
```

Static Similarity Visualization

The cell below generates static similarity maps for several reference points on the image. This is the key deliverable for your PDF submission, since interactive plots don't render in PDFs. We will check just to make sure the visualizations pass the sanity check!

```
In [11]: %matplotlib inline
# Static visualization: pick 4 reference points and show similarity maps
# These pixel coordinates correspond to different semantic regions of the image
reference_points = [
    (259, 200, "Center (face area)"),
    (100, 100, "Top-left (background)"),
    (400, 350, "Bottom-right (body)"),
    (259, 450, "Bottom-center (ground)"),
]

display_img = explorer.original_image.resize((518, 518))

fig, axes = plt.subplots(2, 4, figsize=(20, 10))

for i, (px, py, label) in enumerate(reference_points):
    # Left column: original image with reference point marked
    axes[0, i].imshow(display_img)
    axes[0, i].plot(px, py, 'r+', markersize=25, markeredgewidth=3)
    axes[0, i].set_title(f'Reference: {label}', fontsize=11)
    axes[0, i].axis('off')

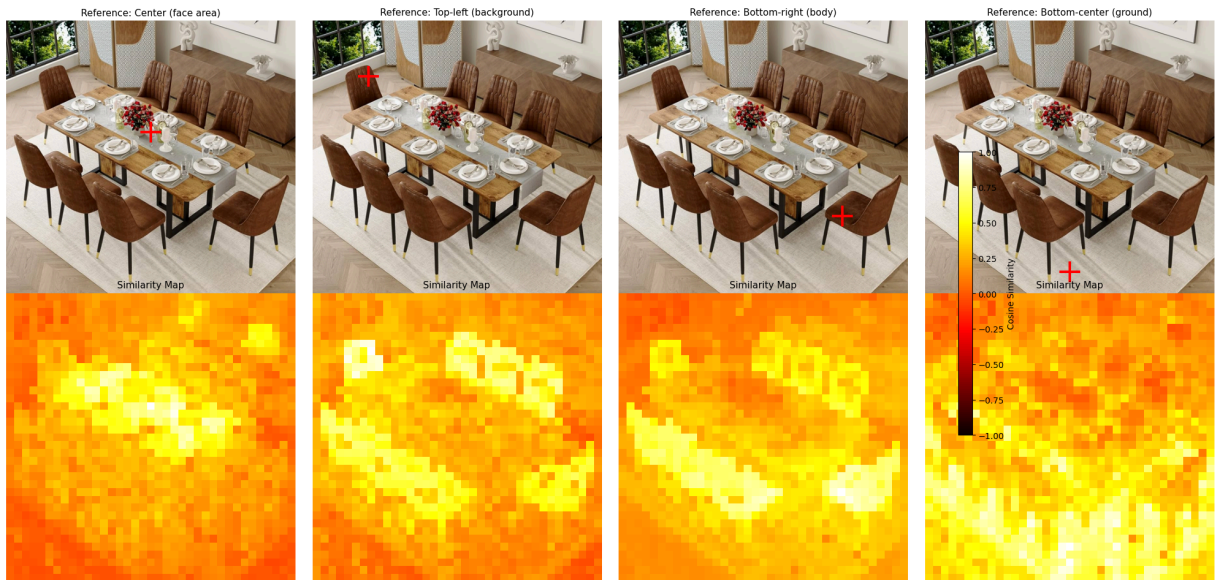
    # Right column: cosine similarity heatmap
    patch_idx = explorer.pixel_to_patch(px, py)
    sim_map = explorer.compute_similarity(patch_idx)
    im = axes[1, i].imshow(sim_map, cmap='hot', vmin=-1, vmax=1)
    axes[1, i].set_title(f'Similarity Map', fontsize=11)
    axes[1, i].axis('off')

# Add a single colorbar
fig.colorbar(im, ax=axes.ravel().tolist(), shrink=0.6, label='Cosine Similarity')
plt.suptitle('DINOv2 Patch Feature Cosine Similarity', fontsize=16, y=1.02)
plt.tight_layout()
plt.savefig('dinov2_similarity.pdf', bbox_inches='tight')
plt.show()
print("Notice how semantically similar regions (e.g., fur patches, background) are highlighted in the similarity maps.")
```

/tmp/ipython-input-262/1492887432.py:32: UserWarning: This figure includes Axes that are not compatible with tight_layout, so results might be incorrect.

```
plt.tight_layout()
```

DINOv2 Patch Feature Cosine Similarity



Notice how semantically similar regions (e.g., fur patches, background areas) light up together!